ATLAS

NAVIGATE THE COMPLEXITY OF DATA

PROJECT ATLAS

Technical Overview and System Guide

Offline Intelligence and Data Operations Platform

Architecture, features, data model, security, operations, testing and project status — everything in the repository, explained in one place.



VERSION	RELEASED	DOCUMENT DATE	LICENSE
0.2.0	28 October 2026	November 2026	MIT

All sample data referenced in this document is synthetic. Figures were measured by the project's own test, acceptance and benchmark runs.

ABOUT THIS DOCUMENT

Contents

This guide explains **Project Atlas** end to end — why it exists, how it is built, what every module does, how it is secured and operated offline, how it was tested, and what is complete, partial or deferred. Facts come directly from the repository: source code, documentation, sample-data ground truth and the recorded test, acceptance and benchmark results for release 0.2.0.

01 Executive Summary . . . 3
02 Vision, Principles and Scope . . . 5
03 Capabilities at a Glance . . . 7
04 System Architecture . . . 9
05 Codebase Tour . . . 12
06 Data Model and Provenance . . . 14
07 Ingestion and Data Quality . . . 16
08 Ontology and Entity Resolution . . . 19
09 Knowledge Graph . . . 21
10 Search and Document Intelligence . . . 23
11 Timeline, Maps and Analytics . . . 25
12 Optional Local AI . . . 28
13 Investigations, Workflows and Reports . . . 29
14 Security Model . . . 31
15 Audit, Backup and Offline Assurance . . . 33
16 Deployment and Operations . . . 35
17 Guided Tour: the Northwind Harbor Demo . . . 37
18 API Reference Overview . . . 39
19 Testing, Validation and Performance . . . 41
20 Status, Limitations and Roadmap . . . 43
A Glossary . . . 45
B Frequently Asked Questions . . . 46

Suggested reading paths

READER	START WITH	THEN
Decision makers	1 Executive Summary, 2 Vision	3 Capabilities, 20 Status and Roadmap
Engineers	4 Architecture, 5 Codebase, 6 Data Model	7–13 feature chapters, 18 API, 19 Testing
Security reviewers	14 Security Model, 15 Audit and Offline	12 Local AI, 20 Limitations
Administrators	16 Deployment and Operations	15 Backup and Restore, 19 Performance
Analysts	17 Guided Tour	7–13 feature chapters

CHAPTER 01

Executive Summary

01

Project Atlas is an offline-first, local-control platform for turning authorized datasets and documents into evidence-backed, auditable analysis — with no cloud dependency at install time or at run time.

Analysts in compliance, auditing, research and organizational review routinely have to combine spreadsheets, database extracts, PDFs and e-mails, work out which records describe the same person or organization, explore how those entities are connected, and then defend their conclusions to someone else. Cloud tools are often ruled out by data sensitivity or air-gapped networks, and ad-hoc local tools lose provenance. Atlas covers the entire chain — import, quality control, entity resolution, graph exploration, search, timelines, maps, analytics, investigations and reporting — inside a single process that runs on a workstation or a private LAN.

167
REST API operations, documented in OpenAPI

16
UI modules in the analytical workstation

111
backend tests passing (plus 11 UI unit tests and an end-to-end run)

13
file formats: 8 tabular, 5 document

13
data-quality checks with plain-language explanations

0
cloud services, telemetry or external fallbacks

What the platform does

- **Import and validate** CSV, TSV, XLSX, JSON, JSONL, XML, SQLite and Parquet through a controlled upload → inspect → map → dry run → commit pipeline with 13 quality checks and versioned, hashed datasets.
- **Resolve entities** with a deterministic, explainable matcher (algorithm *er-1.1*) whose every candidate shows matching, conflicting and “not compared” fields, followed by human review and reversible merges.
- **Explore relationships** in a knowledge graph where each edge is labelled *observed*, *inferred*, *model hypothesis* or *manual*, with centrality, communities, paths and “explain connection”.
- **Search and read** across records, entities, documents, reports and investigations; a grounded document assistant quotes page-level evidence and declines when there is none.
- **Analyse reproducibly** — descriptive statistics, trends, correlation, outliers, time series and a sandboxed SQL console; every run is re-executable and verified by hash.
- **Investigate and report** with verbatim evidence, a reviewed findings workflow, typed visual workflows and PDF/HTML/XLSX/CSV/JSON reports that carry a manifest of exactly what they were built from.
- **Prove what happened** with an append-only, hash-chained audit log, verified (optionally encrypted) backups and an offline verification report.

What makes it different

PROPERTY	HOW ATLAS DELIVERS IT
Offline by design	No external assets, no telemetry, an in-process outbound network guard, bundled basemap and fonts; verified inside a network namespace with no route to the internet.
Evidence over assertion	Original values are kept beside normalized ones; every entity, edge, analysis and report links back to source records and file hashes.
Explainable automation	Matching, quality and graph explanations are deterministic and versioned; scores are labelled as similarity, not probability.
Human in the loop	Merges, findings, model-suggested relationships and generated SQL all require an explicit human decision.
Least privilege	Two-layer RBAC, restricted datasets and sensitive-field redaction enforced by the backend on every route; system administrators do not automatically see project data.
Tamper evidence	SHA-256 hash chain over canonical JSON, database triggers that block edits, HMAC-signed checkpoints.

Status at a glance (release 0.2.0, 28 October 2026)

All automated suites pass: 111 backend tests, 11 frontend unit tests, a Playwright end-to-end run across all modules and the 14-step acceptance scenario including a network-isolated phase. Known gaps are stated plainly: PostgreSQL support is implemented but untested, local-AI features are tested against a mock runtime, Windows/macOS packages have not been built, and no independent penetration test has been performed (Chapter 20).

CHAPTER 02

Vision, Principles and Scope

02

Atlas is built around a small set of principles that show up repeatedly in the code: keep control local, keep the evidence, explain every automated step, and never let software make a judgement a person should make.

The problem it addresses

Organizational analysis rarely starts with clean data. A typical review receives a staff list as CSV, a supplier register in Excel, a payments extract from a database, a handful of audit reports as PDF and a folder of e-mails. The same person appears as *Dr. Amelia Okafor* in one file and *Amelia Okafor* in another; a phone number is stored in three formats; timestamps arrive in local time without a zone. The analyst must reconcile all of this, spot what is unusual, and produce a conclusion that will be challenged — often on a machine that may not connect to the internet.

General-purpose BI tools do not keep row-level provenance or support entity resolution; cloud analytics platforms are unavailable for sensitive or air-gapped work; scripts and spreadsheets cannot show a reviewer how a conclusion was reached. Atlas is designed to close that gap with one installable product.

Design principles

PRINCIPLE	WHAT IT MEANS IN PRACTICE	WHERE IT IS ENFORCED
Offline-first	Install from a bundle and run with no network; private-range services only.	`core/netguard.py`, `core/config.py`
Local control	One process, local SQLite or a LAN PostgreSQL; data never leaves the host.	deployment profiles
Evidence first	Keep raw values, hashes and record links at every step.	`records.raw`, `entity_sources`
Explainability	Deterministic, versioned algorithms with visible inputs.	`er-1.1`, `quality-1.0`
Human judgement	Software proposes; people decide and are recorded.	review, findings, AI
Reproducibility	Re-run any analysis on recorded inputs and compare hashes.	`analysis_runs`
Least privilege	Deny by default; hide existence of other projects (404).	`security/rbac.py`
Tamper evidence	Append-only audit with a hash chain and checkpoints.	`audit/service.py`
Honest status	Document what is untested or deferred rather than imply it works.	engineering report

Product identity

ITEM	VALUE
Product name / short name	PROJECT ATLAS / Atlas — “Navigate the Complexity of Data.”
Python distribution / import package	`project-atlas` / `project_atlas` (Python 3.11+)
API service	`atlas-api` — FastAPI, OpenAPI served at `/api/docs`
UI package	`project-atlas-ui` — React + TypeScript + Vite
Desktop app	Project Atlas — PyInstaller bundle with a chromeless browser window
Command-line tools	`atlas` (serve, create-admin, backup, verify-backup, offline-check, migrate), `atlas-desktop`
Configuration prefix	`ATLAS_` environment variables, overlaid on `.env`
License	MIT; bundled DejaVu fonts and Natural Earth data keep their own licenses

Intended use and responsible-use safeguards

Atlas is intended for authorized research, compliance, auditing and organizational analysis over data the user is permitted to process. Several safeguards are built into the product rather than left to policy:

- Outlier results always carry a disclaimer that statistical unusualness is *not evidence of wrongdoing*.
- Connection paths in the graph “show recorded links only; they do not establish intent or wrongdoing”.
- Temporal proximity between two entities is described as descriptive, not proof of interaction.
- Investigations require an objective and a scope/authorization basis; findings need independent review.
- All bundled demonstration data is **synthetic** (seed 20251027, e-mail domain *example.invalid*, `record_origin=SYNTHETIC` on every row).

Who uses it

ROLE	TYPICAL WORK IN ATLAS
System Administrator	Users, projects, backups and restore, settings, local model registry, audit read
Security Administrator	Account security, session revocation, MFA resets, audit read and chain verification
Project Administrator	Membership, access policies, sensitive data, dataset deletion, project archive
Data Engineer	Imports, ontology, restricted datasets, entity resolution, data-writing workflows
Analyst	Analysis, SQL, graph and map annotations, investigations, reports, AI assistants
Reviewer	Match review, finding review, read access
Read-Only User	Browse and search

CHAPTER 03

Capabilities at a Glance

03

The user interface is an analytical workstation with sixteen modules. Each is backed by real API behaviour, access control and automated tests.

MODULE	KEY	WHAT YOU DO THERE
Atlas Home	g h	Project dashboard: counts, events over time, entities by type, most-connected entities, data quality by check, activity feed, watchlist, running jobs
Data Catalog	g c	Datasets, versions, normalized vs original values, schema completeness, quality issues and explanations, lineage, access and retention policy
Ingestion	g i	Upload, inspect, map fields, define entities/links/events/map points, dry run, commit as a background job
Global Search	g s	Boolean, phrase, prefix, field and numeric/date filters across all permitted content; saved searches and history
Entity Explorer	g e	Entities with attribute provenance, match review, merge history and revert, watchlists
Knowledge Graph	g g	Neighbourhoods, filters by type and basis, centrality, communities, paths, explain connection, snapshots, exports
Timeline	g t	Events in a chosen display time zone, gap clustering, relationships in a window, two-entity comparison
Geospatial Explorer	g m	Offline basemap, layers, heatmap and clustering, radius and distance tools, annotations, spatial joins
Analytics	g a	Reproducible analyses, run history and reproduction, read-only SQL console, “Ask your data”
Documents	g d	PDF/DOCX/TXT/MD/EML processing, page view, similar documents, grounded document assistant
Investigations	g v	Scope, collaborators, linked items, verbatim evidence with revisions, findings and review
Workflows	g w	Typed visual pipelines: validate, version, preview without side effects, run with confirmation
Reports	g r	Compose sections, preview, export PDF/HTML/XLSX/CSV/JSON with a manifest
Audit Center	g u	Filter events, verify the hash chain, write signed checkpoints, export CSV
Administration	g x	System and offline status, users, members, ontology, backups, retention, AI models
Settings	g ,	Profile, password, MFA enrolment, sessions, theme and display preferences

Platform capabilities

AREA	HIGHLIGHTS
Identity & access	Argon2id, one-time setup token, sliding sessions with revocation, rate limiting and lockout, optional TOTP MFA, admin-issued recovery tokens, global and project roles, restricted datasets, sensitive-field redaction
Ingestion & catalog	8 tabular formats, encoding/delimiter/type detection, normalization (time zones, numbers, phones, e-mails, case), dry runs, 13 quality checks, versioned datasets with lineage

AREA	HIGHLIGHTS
Ontology & resolution	10 default entity types and 9 relationship types, editable; explainable matching with evidence; human review; reversible merges
Knowledge graph	Typed, directional edges with assertion basis; neighbourhoods, paths, components, centrality, Louvain communities, snapshots, GraphML/GEXF/JSON export
Search & documents	FTS5 with BM25 and snippets; PDF/DOCX/TXT/MD/EML extraction, language, keywords, entity candidates, duplicates, page-level evidence, grounded assistant
Time & space	UTC event times with source text and zone; offline MapLibre maps; radius, bbox, distance, spatial join, heatmap, clustering
Analytics	Describe, frequency, trend, correlation with confidence intervals, outliers, missing data, time series; reproducible by hash; sandboxed DuckDB SQL
Optional local AI	Grounded answers, SQL drafts, summaries, schema-mapping suggestions, relationship hypotheses, embeddings and semantic search — local endpoint only
Case work	Investigations with evidence revisions; findings state machine; workflows with rollback; reports with manifests
Assurance	Hash-chained audit, checkpoints, encrypted backups with verification and restore, offline verification report
Usability (0.2)	One-click synthetic demo, Ctrl/ K command palette, saved views, watchlists and notifications, explain connection, retention enforcement

CHAPTER 04

System Architecture

04

Atlas is a modular monolith: one Python process serves the REST API, runs background jobs and serves the built React UI. Extra services are introduced only where they are genuinely required.

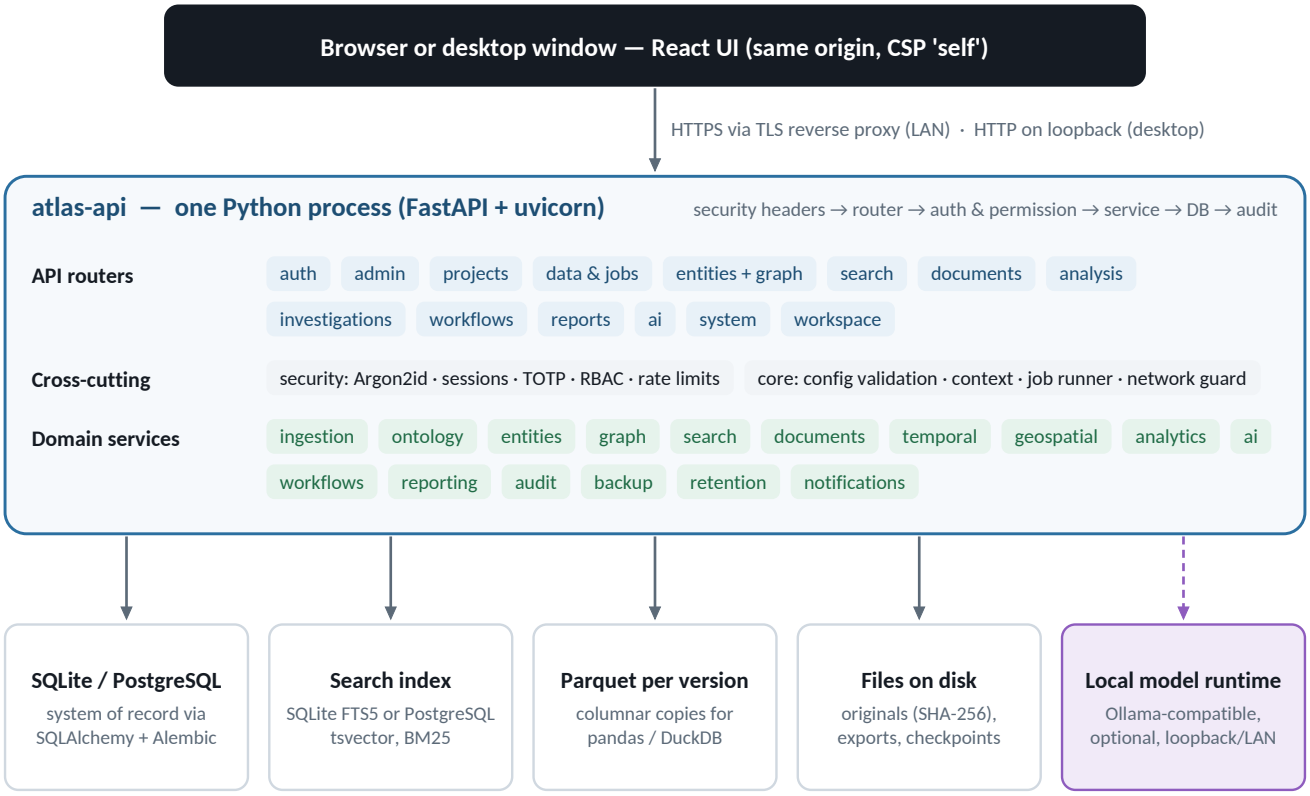


***Figure 1** — Component view. Routers translate HTTP into calls on domain services; all services share one database session factory, job runner, audit log and network guard. The optional model runtime is the only external process Atlas talks to.*

Why a modular monolith

A single process keeps installation, upgrades and the security boundary simple: one runtime, one set of dependencies, one place where authorization is enforced, and one audit log. Packages are separated by domain (`ingestion`, `entities`, `graph`, `search`, `analytics` and so on) and communicate through plain Python interfaces, so a service can be extracted later if measurements justify it. The engineering report explicitly defers a dedicated graph database, an external search engine, a vector database and multi-process workers because the measured workloads do not need them.

Responsibilities of each store

STORE	RESPONSIBILITY	WHY THIS CHOICE
SQLite (single user) / PostgreSQL (LAN), SQLAlchemy + Alembic	System of record: users, sessions, projects, datasets and versions, records (raw + normalized JSON), entities, relationships, documents, investigations, workflows, reports, audit	Transactions, constraints and migrations; SQLite needs no server and backs up online
`search_index` (FTS5 / tsvector)	Full-text search over records, entities, documents, reports and investigations with access tags	Built in, no extra service; BM25 ranking and snippets
Parquet per dataset version	Columnar copy of normalized records for analytics, SQL and workflows	Fast scans with pandas, DuckDB and Arrow; immutable per version, so reproducible
DuckDB (in memory, per query)	Read-only SQL console	Isolated: only permitted tables loaded, external access disabled
NetworkX (in memory, per request)	Graph algorithms over relationships	Adequate to about 10^5 edges on a workstation
Filesystem (`files/`, `exports/`)	Uploaded originals (never modified), report exports with manifests	Originals stay byte-identical; SHA-256 recorded

Life of a request

Every API call passes through the same sequence. A middleware adds security headers (CSP, frame denial, no-referrer, COOP, permissions policy and `Cache-Control: no-store` for API responses). The router validates the payload with Pydantic. Dependencies in `api/deps.py` resolve the bearer token to a session, enforce the per-user rate limit, load the caller's global role and project membership, and check the permission the route declares. Non-members of a project receive 404 for every project URL, so the existence of a project is never revealed, and the denial is audited. The domain service then runs inside a SQLAlchemy session, and any security-relevant action appends an audit event in its own transaction — denials are recorded even when the request fails. Errors are returned in one shape: `{"error": {"code", "message", "details"}}`.

Background jobs

Long operations (imports, entity resolution, document processing, embeddings, workflow runs, demo creation) are persisted in the `jobs` table and executed by a small thread pool, or inline when `ATLAS_JOBS_INLINE=true`. Because SQLite allows one writer, live progress is kept in memory and persisted when the job ends; clients follow it over server-sent events at `/api/jobs/{id}/events` or by polling. Jobs support cancellation and retry, and jobs interrupted by a restart are marked failed and retryable. Imports are atomic, so a retry always starts cleanly.

Deployment profiles

The same code serves three profiles. Startup validation refuses unsafe combinations: a non-loopback binding without LAN opt-in, a LAN production deployment without TLS, a non-local AI endpoint in offline mode, or telemetry.

Standalone desktop

BINDING
127.0.0.1, random port

DATABASE
SQLite in the user data directory

NOTES
PyInstaller bundle opens a chromeless Edge/Chrome/Chromium window

Air-gapped workstation

BINDING
127.0.0.1:8000

DATABASE
SQLite

NOTES
installed from the offline bundle (wheelhouse + SHA256SUMS)

Private LAN server

BINDING
behind a TLS reverse proxy

DATABASE
PostgreSQL recommended (preview)

NOTES
ATLAS_ALLOW_LAN + ATLAS_TLS_TERMINATED in production; nginx + systemd examples

Figure 2 — The three supported deployment profiles.

Technology stack

LAYER	COMPONENTS (MINIMUM VERSIONS FROM THE MANIFESTS)
Web/API	FastAPI 0.115, uvicorn 0.30, Pydantic 2.7, python-multipart
Persistence	SQLAlchemy 2.0, Alembic 1.13, SQLite FTS5; psycopg 3 for the optional PostgreSQL profile
Data	pandas 2.2, PyArrow 16, DuckDB 1.0, openpyxl 3.1, tzdata
Graph & matching	NetworkX 3.3, RapidFuzz 3.9
Documents & reports	pypdf 4.2, python-docx 1.1, ReportLab 4.2 with bundled DejaVu fonts
Security	argon2-cffi 23.1, cryptography 42 (AES-256-GCM, scrypt), httpx 0.27 for the local model runtime
Frontend	React 18.3, TypeScript 5.6, Vite 5.4, Tailwind CSS 3.4, React Router 6.28, TanStack Query 5.59, Zustand 5
Visualization	ECharts 5.5, Cytoscape.js 3.30, MapLibre GL 4.7 with a bundled GeoJSON basemap, React Flow (@xyflow/react 12)
Testing	pytest 8, Vitest 2.1 with Testing Library and jsdom, Playwright (Chromium) end-to-end
Packaging	setuptools wheel with the UI bundled, offline wheelhouse bundle, PyInstaller desktop app

Desktop shell decision

Tauri requires WebKitGTK/WebView2 toolchains and Electron adds roughly 150 MB and a second runtime. The shipped desktop shell is therefore a PyInstaller bundle (Python, all dependencies and the UI) that starts atlas-api on a random loopback port with offline mode on and opens a chromeless Edge, Chrome or Chromium window (falling back to the default browser). This keeps one runtime and one security boundary; a native wrapper remains a future option.

CHAPTER 05

Codebase Tour

05

The repository holds a Python backend, a React frontend, packaging and deployment assets, scripts, synthetic sample data and documentation — about 21,000 lines of first-party code and tests.

```
project-atlas/
├── backend/project_atlas/  api, core, security, database (+ migrations), ingestion,
│                           ontology, entities, graph, search, documents, temporal,
│                           geospatial, analytics, ai, workflows, reporting, audit,
│                           backup, demo, cli, desktop
├── backend/tests/          pytest suite (11 modules, 111 tests)
├── frontend/               React UI: src/app, components, features/*, services, types
├── desktop/                PyInstaller spec, launcher, icons
├── deployment/             offline installers, nginx and systemd examples
├── scripts/                sample data, acceptance scenario, benchmark, builds
├── sample-data/            SYNTHETIC demo data + expected_results.json
└── docs/                   architecture, install, offline, security, data model,
                            user guide, backup, API, release notes, engineering report
```

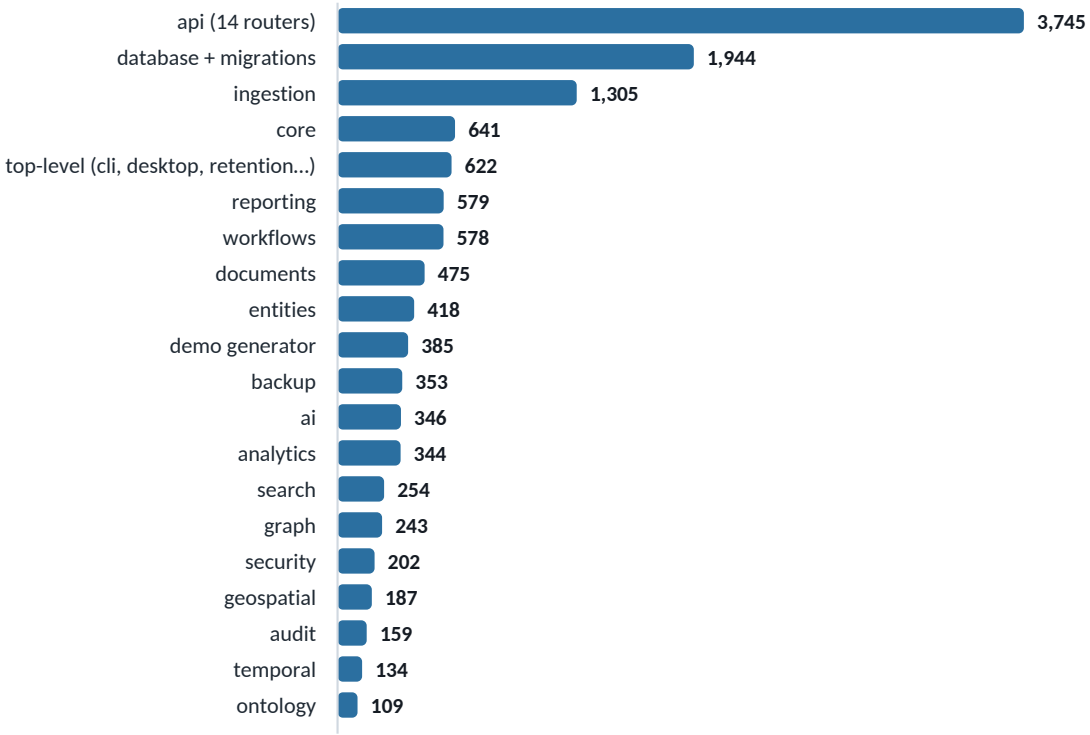


***Figure 3** — Backend size by package (lines of Python, total ≈ 13,000). The API layer is the largest because each router carries its own validation, permission checks and response shaping.*

Backend packages

PACKAGE	RESPONSIBILITY	NOTABLE FILES
api	HTTP routers, request models, dependencies (auth, permission, project scoping)	`deps.py`, `entities.py`, `data.py`
core	Settings and validation, application context, job runner, network guard, errors	`config.py`, `jobs.py`, `netguard.py`
security	Argon2id passwords, TOTP, RBAC tables, rate limiting	`rbac.py`, `passwords.py`
database	ORM models (≈ 50 tables) and Alembic migrations 0001–0004	`models.py`, `migrations/`
ingestion	Parsers, normalization, quality checks, import pipeline	`parsers.py`, `quality.py`
ontology / entities	Type definitions; matching algorithm, review, merge and revert	`matching.py`, `service.py`
graph	Neighbourhoods, algorithms (own PageRank), exports	`graph/service.py`
search / documents	Query parser and FTS; extraction, language, keywords, chunks, assistant	`text.py`
temporal / geospatial	Event queries, clustering, comparison; geometry, radius, joins	`service.py`
analytics	Analysis kinds, run recording and reproduction, SQL sandbox	`sql.py`
ai	Local model client, registry health, grounded generation, embeddings	`ai/service.py`
workflows / reporting	Node catalog and executor; report sections and five renderers	`engine.py`
audit / backup	Hash-chained log and checkpoints; archive, verify, restore, scheduler	`backup/service.py`
demo	Deterministic synthetic Northwind Harbor generator and loader	`generator.py`

Frontend structure

The UI (≈ 4,400 lines of TypeScript) is organized by feature. `src/app` holds the shell (`Layout.tsx` with navigation, top bar, tabs and keyboard shortcuts), routing and the Zustand store for UI state (theme, tabs, panel sizes, recent projects). `src/services/api.ts` wraps fetch with the bearer token kept in `sessionStorage`; server state is cached with TanStack Query. `src/components` contains the shared component layer (tables, charts, dialogs and toasts, the command palette) and `src/features/*` holds one folder per module. Light and dark themes use CSS variables; the build is served by atlas-api from the same origin under a strict CSP.

Scripts and tooling

SCRIPT	PURPOSE
`scripts/generate_sample_data.py`	Regenerates the synthetic dataset deterministically (seed 20251027)
`scripts/acceptance_scenario.py`	14-step release scenario, including a phase inside a network namespace with loopback only
`scripts/benchmark.py`	Performance measurements at configurable scale (100,000 rows / 5,000 people in the report)
`scripts/build_release.sh`	Offline bundle: project wheel with UI, dependency wheelhouse, installers, SHA256SUMS
`scripts/build_desktop.sh`	PyInstaller desktop application for the current OS
`scripts/generate_api_docs.py`	Regenerates docs/api.md and openapi.json from the running application

CHAPTER 06

Data Model and Provenance

06

The schema is the system of record for everything Atlas knows, including how it came to know it. Provenance is not a feature bolted on top; it is part of the primary keys and foreign keys.

Identity
users · sessions · password_reset_tokens

Projects
projects · project_members · settings

Data
data_sources · files · datasets · dataset_versions · records · quality_issues · jobs

Ontology
entity_types · relationship_types

Entities & graph
entities · entity_sources · relationships · match_candidates · merge_history · graph_snapshots

Documents
documents · document_chunks · embeddings

Time & space
events · geo_features

Search
search_index (FTS5 / tsvector) · saved_queries · query_history

Analysis
analysis_runs (inputs, versions, result SHA-256)

Investigations
investigations · collaborators · items · notes · evidence (+revisions) · findings

Workflows & reports
workflows · workflow_versions · workflow_runs · reports · report_exports

Platform
audit_events (hash chain) · model_registry · backup_manifests · saved views · watches · notifications

Figure 4 — Functional areas of the schema (source of truth: backend/project_atlas/database/models.py).

Conventions

- Primary keys are stable prefixed UUIDs — `prj_…`, `dst_…`, `rec_…`, `ent_…`, `mrg_…` — so an identifier tells you what it refers to.
- The audit log uses an integer sequence instead, so the hash chain has a total order.
- JSON columns hold flexible payloads: raw and normalized records, entity attributes with per-value provenance, match evidence, workflow definitions, report manifests.
- Indexes cover the filters that matter: foreign keys, (project, entity type), (project, event time), (dataset version, row index), identity keys and audit actions.
- Alembic migrations are applied automatically at startup and restores migrate older backups forward: 0001 initial schema, 0002 search index and audit append-only triggers, 0003 indexed FTS rowid map (performance), 0004 saved views, watches and notifications.

Key tables

TABLE	WHAT IT STORES
`datasets / dataset_versions`	Name, owner, access policy, retention policy; immutable versions with schema, import mapping, record count, quality summary, ingestion report, content SHA-256, origin (import or workflow) and parent version
`records`	Raw source values and normalized values as separate JSON, row index and row hash
`entities`	Type, display name, deterministic `identity_key`, attributes as `{field: [{value, normalized, record_id, dataset_version_id}]}`, status active/merged
`relationships`	Source and target, type, assertion (observed / inferred / model_suggested / manual), status (active / unverified / rejected), confidence (null unless measured), evidence, valid_from/to
`match_candidates / merge_history`	Scored pairs with evidence and review status; merge snapshots for exact reversal
`analysis_runs`	Kind, parameters, inputs (version ids and hashes, or a graph-state hash), algorithm and software version, result and result SHA-256
`evidence / findings`	Verbatim excerpts with location and revision history; findings with a status state machine and supporting/contradicting links

Provenance and lineage

Every derived object can be traced back to bytes on disk: versions record the file hash and mapping, records their row hash, attributes their source record, relationships their evidence, analyses their input hashes, and report manifests everything a report was built from.

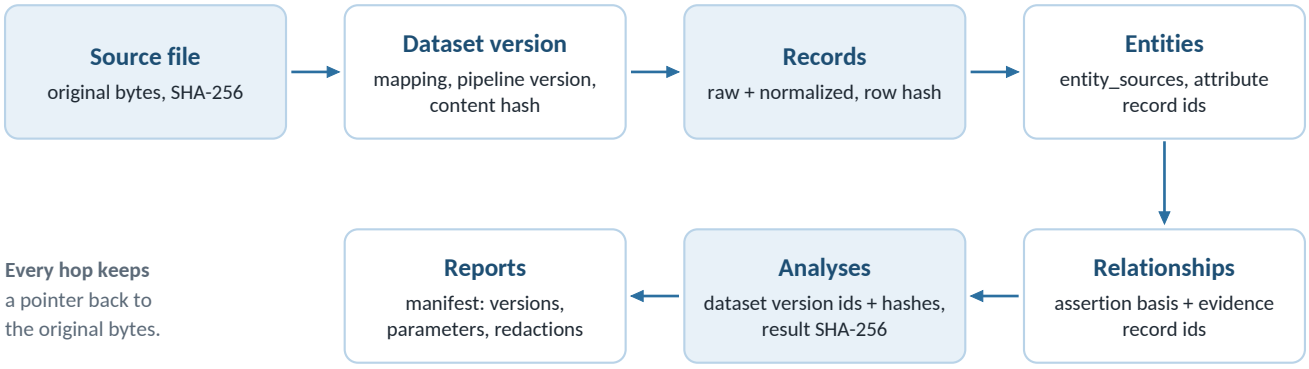


***Figure 5** — The provenance chain from source file to report. Workflow-materialized versions also record their parent version and workflow run.*

Deletion and retention policy

- Dataset contents (records, versions, quality issues, derived events, observed relationships, search entries) are deleted only by *Delete dataset* (`dataset.delete` plus typing the name); entities still supported elsewhere remain.
- Project deletion cascades and is limited to system administrators who administer the project (typed confirmation). Merges are never destructive, audit events are never changed, and evidence survives document deletion.
- Retention policies (`indefinite`, `until_case_closed`, `1y`, `3y`, `7y`) are enforced from 0.2: superseded versions past retention are purged (tombstones keep lineage); current versions are never auto-deleted.

CHAPTER 07

Ingestion and Data Quality

07

Data enters Atlas through one controlled pipeline. Nothing is written until the user has seen a dry run over the whole file, and the commit is all-or-nothing.

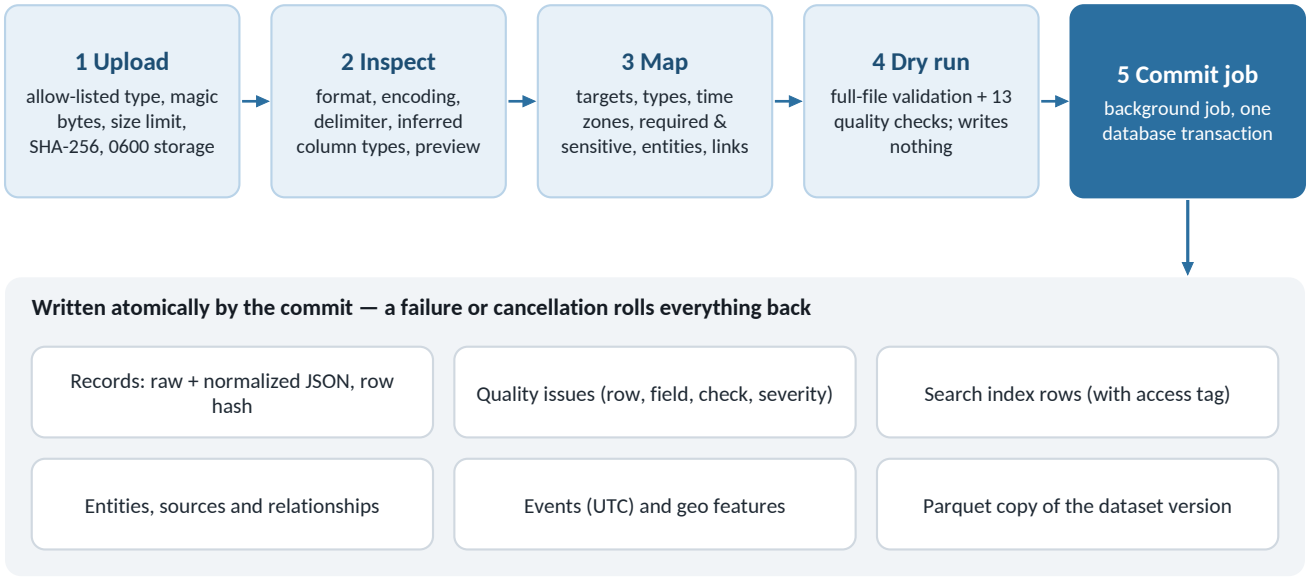


Figure 6 — The import pipeline and what a commit writes.

Supported formats

FORMAT	HANDLING
CSV / TSV	Encoding detection (UTF-8, cp1252, latin-1 with mojibake detection), delimiter sniffing, duplicate header renaming, extra-column detection
XLSX	Sheet selection via openpyxl; ZIP magic-byte check
JSON / JSONL	Record path selection and flattening of nested objects into dotted columns
XML	Record element selection; DTD and ENTITY declarations rejected to prevent entity-expansion attacks
SQLite	Opened read-only (`mode=ro`); table names validated against the catalog
Parquet	Magic-byte check; read through PyArrow
PDF, DOCX, TXT, Markdown, EML	Processed as documents rather than tabular data (Chapter 10)

Inspection, mapping and normalization

Inspection reports the detected format, encoding, delimiter, inferred column types and a preview. The mapping step turns source columns into typed target fields. Supported types are string, identifier, integer, float (with unit suffix detection), boolean, date, datetime (with a source time zone and optional format), e-mail, phone, latitude and longitude. Each field can be marked *required* or *sensitive*, given a case rule or a validation pattern, and configurable null tokens are recognized. Atlas suggests a mapping automatically, and with a local model can propose names, types and sensitivity, which are validated before being applied.

Crucially, the **original value is stored separately from the normalized value**. The catalog lets anyone toggle between the two, so a reviewer can always see exactly what the source said. Datetimes are converted to UTC while the original text and zone are kept; daylight-saving transitions are handled through the IANA database.

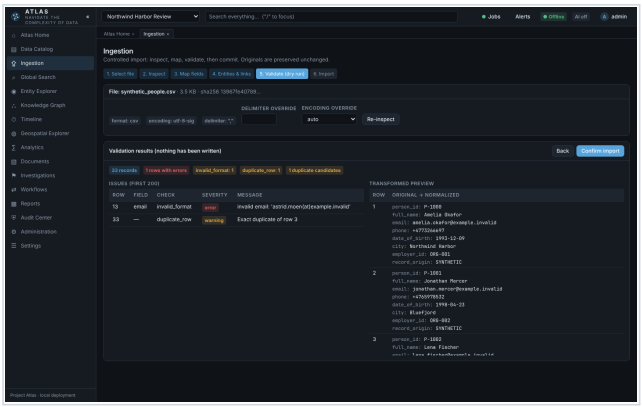
Deriving entities, links, events and map points

The same mapping can declare that each row produces entities (for example a *Person* keyed by `person_id` and an *Organization* keyed by `employer_id`), relationships between them (*works for*, *transacted with* valid from an event time), timeline events and map points. Rows with invalid coordinates produce no map point — Atlas never invents coordinates. Event titles use simple field substitution only, never format-string attribute access.

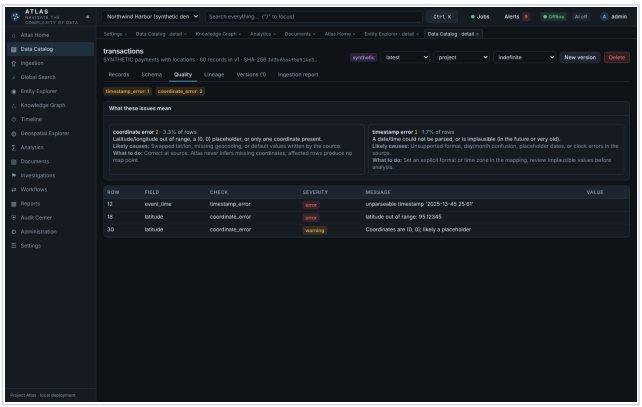
The 13 quality checks

CHECK	SEVERITY	WHAT IT DETECTS
`parse_error`	error	Row could not be parsed
`extra_columns`	warning	Row has more values than header columns
`encoding`	warning	Replacement characters or mojibake such as “Ã©”
`invalid_format`	error	Value does not match its declared type or pattern (e.g. a malformed e-mail)
`timestamp_error`	error / warning	Unparseable date or time, a timestamp in the future, or before a minimum year
`coordinate_error`	error / warning	Latitude/longitude out of range, (0, 0) “null island”, or only one of the pair
`unexpected_type`	error	Numeric or boolean field holds text
`missing_required`	error	A required field is empty
`duplicate_row`	warning	Exact duplicate of an earlier row (by row hash)
`duplicate_key`	warning	Unique key value already used by another row
`referential_integrity`	error	Value not found in a referenced dataset (e.g. an unknown organization id)
`inconsistent_units`	warning	Mixed unit suffixes within one numeric column
`schema_drift`	warning	Fields added, removed or changed type compared with the previous version

Each issue records the row, field, check, severity, message and the offending value (capped at 500 issues per check for large files). The summary includes per-field completeness and the number of rows with errors. Every check has a deterministic plain-language explanation — what it means, likely causes and what to do — shown with or without AI; a local model can optionally add a prioritized summary.



Dry run over the transactions file: planted timestamp and coordinate errors are reported before anything is written.



Quality explanations in the Data Catalog: meaning, likely causes and recommended action per check.

***Figure 7** — Ingestion dry run and plain-language quality explanations.*

Versions, lineage and jobs

Re-importing a dataset creates a new immutable version; schema drift is reported against the previous one. The catalog shows lineage from source file hash through mapping to derived entities and relationships, plus the full ingestion report. Commits run as background jobs in a single transaction with batched inserts; failure or cancellation rolls everything back and *Retry* re-runs the import from the start. Data engineers can switch datasets to *restricted*; project administrators can delete them.

CHAPTER 08

Ontology and Entity Resolution

08

Entity resolution decides which records describe the same real-world person or organization. Atlas treats it as a decision support problem: the algorithm explains itself, and a person makes every merge.

A configurable ontology

Each project starts with a default ontology that data engineers can extend. Entity types define fields with a type, an *identifier* flag (used for deterministic matching), a *sensitive* flag (drives redaction), a *required* flag and an optional pattern; the type carries a schema version. Relationship types declare direction and the permitted source and target types.

DEFAULT ENTITY TYPES	IDENTIFIER FIELDS (SENSITIVE IN ITALICS)
person	*email*, *phone*, *national_id* (also sensitive: *date_of_birth*)
organization	registration_number
account	*account_number*
transaction	transaction_id (plus amount, currency, event_time)
location · event · document · asset · device · project	document_id, serial_number, device_id, code

Default relationship types: *works for* (person → organization), *located in*, *owns*, *transacted with*, *associated with* (undirected), *participated in*, *mentioned in*, *sent payment* and *received payment*.

Algorithm er-1.1

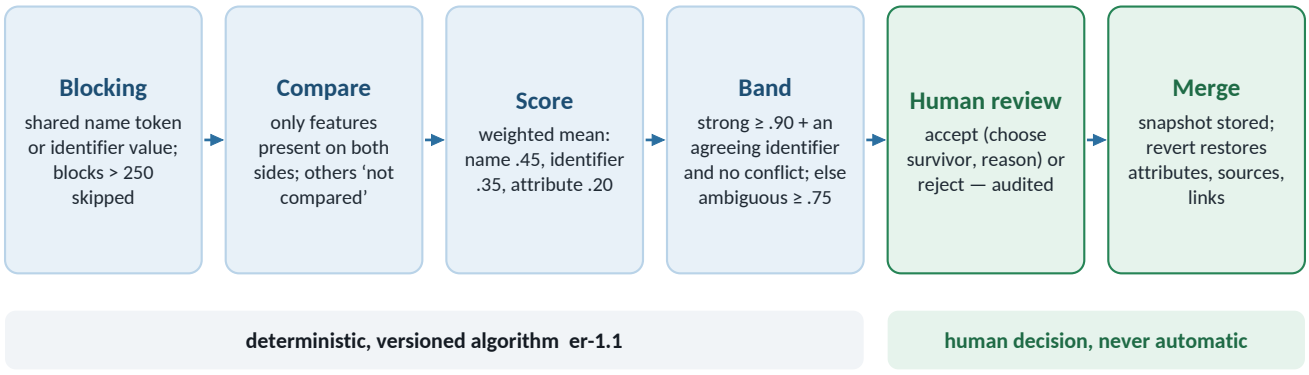


***Figure 8** — Entity resolution: deterministic candidate generation and scoring, then human decision.*

- **Normalization** — Unicode decomposition, accents and punctuation removed, lower-cased; honorifics (Dr, Mr, Prof…) dropped for people and legal suffixes (Ltd, GmbH, Inc…) for organizations.
- **Blocking** — candidate pairs must share a normalized name token or an identifier value; oversized blocks (more than 250 members) are skipped to bound the work.
- **Name similarity** — the larger of RapidFuzz token-sort ratio and 0.95 × token-set ratio.

- **Identifiers** — exact overlap of normalized values (1 or 0). **Other attributes** — equality, or fuzzy similarity counted only when ≥ 0.9.
- **Missing data is never agreement** — a field missing on either side is listed as “not compared”.
- **Bands** — *strong* needs a score ≥ 0.90, at least one agreeing identifier and no conflicting identifier; otherwise *ambiguous* if the score ≥ 0.75. Matches resting on names and descriptive attributes alone can never be strong. Scores are similarity measures, not calibrated probabilities.

Worked example: the Northwind Harbor demo

PAIR	OUTCOME	WHY
Dr. Amelia Okafor ↔ Amelia Okafor	strong	Same e-mail identifier; honorific removed before name comparison
Jon Mercer ↔ Jonathan Mercer	strong	Same phone identifier outweighs the shortened first name
Lena Fisher ↔ Lena Fischer	ambiguous	Similar names only — needs human judgement
Rafael Ortiz ↔ Rafael Ortiz (contact)	not strong	Phone identifiers conflict, so identical names cannot make it strong
Gunnar Thorsen	no match	No other record shares a token or identifier

These are exactly the outcomes recorded in *sample-data/expected_results.json*, and the test suite asserts them (2 strong, 1 ambiguous, the conflict not strong, no false match, no duplicate candidates on re-run).

Review, merge and revert

Each candidate card lists the compared features with per-feature similarity and method, the conflicting fields and the fields not compared. A reviewer chooses which entity survives and records a reason (audited), or rejects the pair. A merge moves attributes, sources and relationships to the survivor and stores a complete snapshot in `merge_history`; *Revert* restores attributes, sources and relationships exactly, and the UI offers *Undo* straight from the confirmation toast. Users can also **watch** an entity and be notified when a later import adds source records about it.

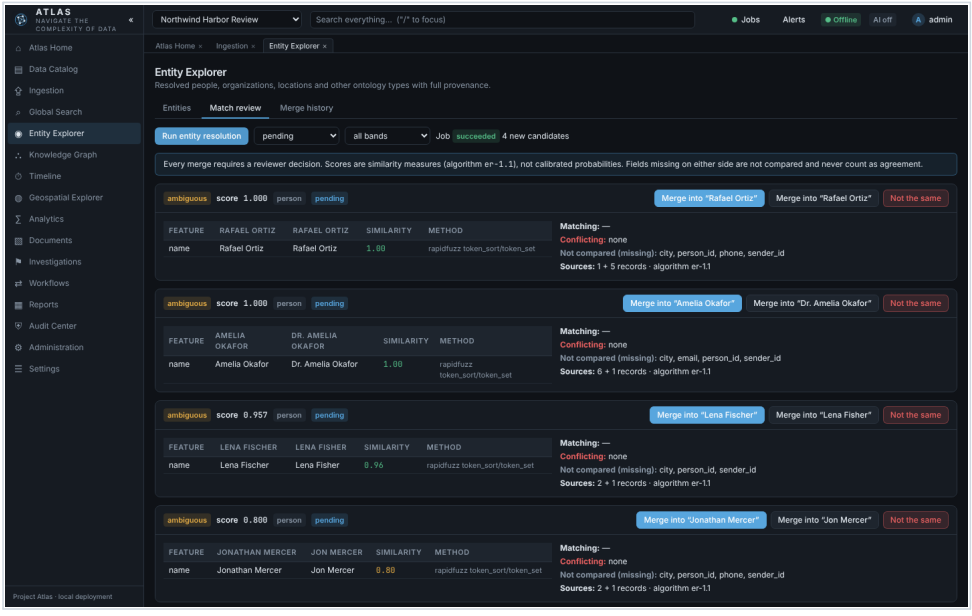


Figure 9 — Match review: compared features, similarities, conflicts and not-compared fields, with merge and reject actions.

CHAPTER 09

Knowledge Graph

09

The graph is where resolved entities and their relationships become explorable. Its defining feature is honesty about where every edge came from.

Every edge has a basis

ASSERTION BASIS	MEANING	LINE STYLE	DEFAULT VISIBILITY
observed	Stated directly by a source record (e.g. employer_id on a person row)	solid	shown
inferred	Deterministic inference from data, e.g. a mapping rule	dashed	shown
model_suggested	Hypothesis proposed by a local model from a document; quote verified	dotted	hidden until verified (`unverified`)
manual	Created by a user, with the user recorded	long dash	shown

Relationships also carry a status (active, unverified, rejected), evidence (record ids and excerpts), optional validity dates and a confidence that is left **null unless it was actually measured**. Reviewers verify or reject hypotheses; verification keeps the original provenance.

Exploration at scale

The overview loads the best-connected nodes first, ranking degrees in SQL instead of materializing the whole graph (this change cut the overview from 6.6 s to 0.9 s on 99,767 edges). Users double-click to focus a node, choose a depth, filter by entity type, relationship type and basis, and opt in to unverified hypotheses. Node limits are capped by `ATLAS_GRAPH_NODE_LIMIT` (default 2,000) and the UI reports “showing N of M nodes”.

ALGORITHM	IMPLEMENTATION NOTES
Degree centrality	From the filtered subgraph
PageRank	Dependency-free power iteration (α = 0.85), verified against NetworkX's reference implementation; removed a SciPy dependency
Betweenness centrality	Exact on small graphs; sampled with k = 500 above 2,000 nodes
Communities	Louvain modularity
Connected components	Weak components over the undirected view
Shortest path	Between two shift-selected nodes

Every algorithm run is stored like an analysis, with a hash of the graph state it ran on, so results can be cited and reproduced. Users can save **snapshots** (node and edge ids, layout and filters) and export subgraphs as GraphML, GEXF or JSON for other tools.

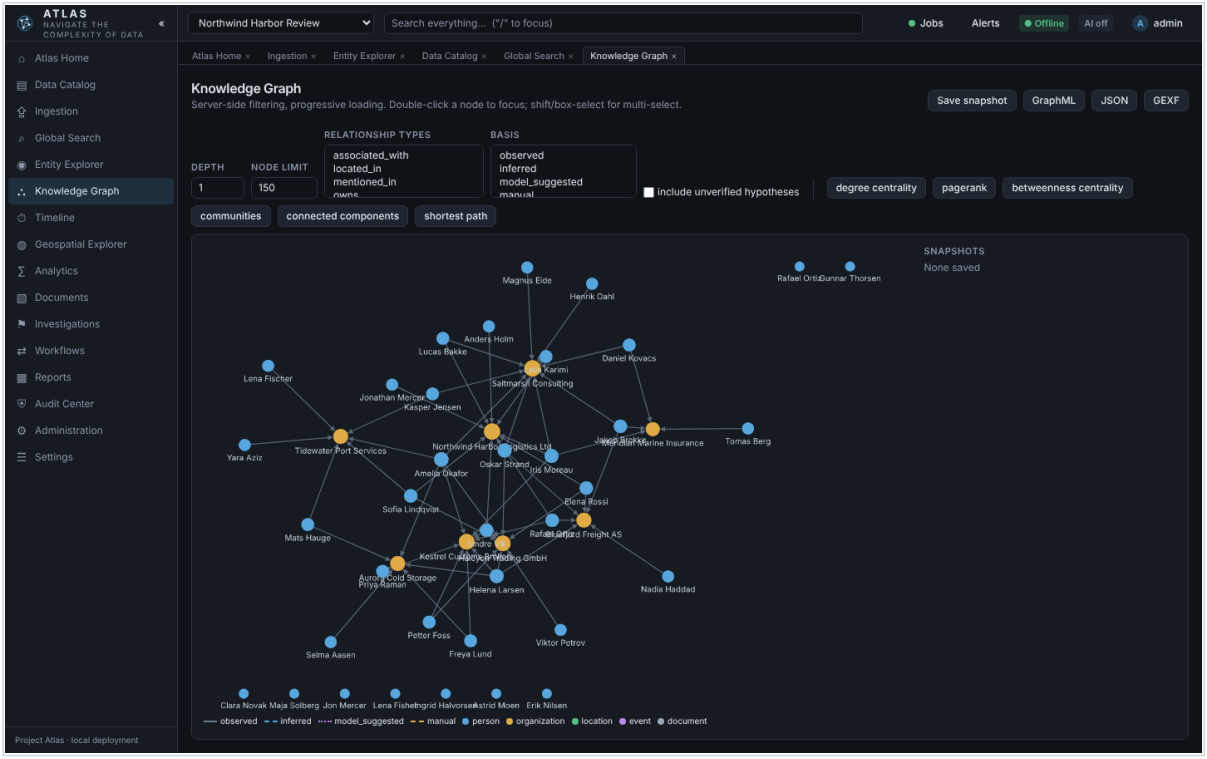


Figure 10 *— Knowledge Graph module: filters by relationship type and basis, algorithm buttons and the legend encoding basis by line style and entity type by colour.*

Explain connection

Selecting two nodes and choosing *Explain connection* returns the shortest paths between them with, for every hop, the relationships involved, their basis and a redacted sample of the source evidence. The answer is framed carefully: a path shows recorded links only and does not establish intent or wrongdoing.

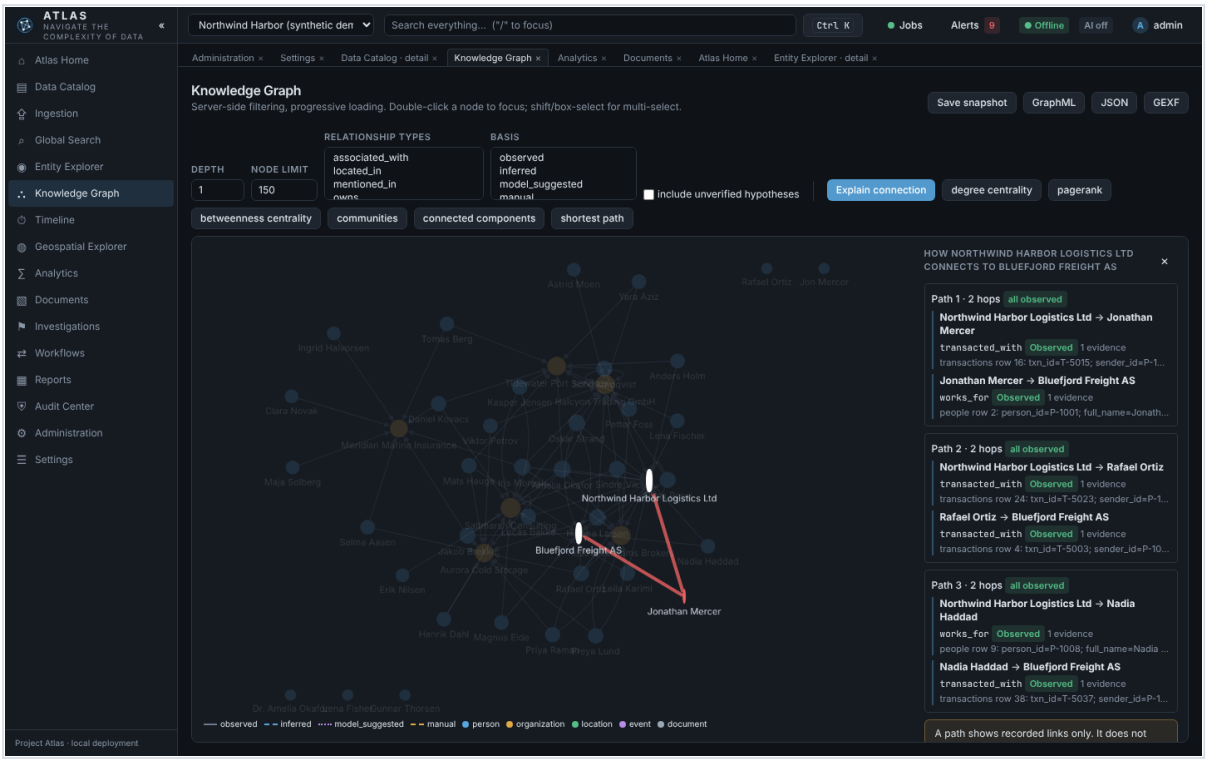


Figure 11 *— Explain connection: per-hop relationships, basis and source evidence between two selected entities.*

CHAPTER 10

Search and Document Intelligence 10

Search reaches everything a user is allowed to see. Document intelligence turns PDFs, Word files and e-mails into page-level evidence that can be quoted, cited and linked to entities.

Unified full-text search

Records, entities, documents, reports and investigations are indexed into `search_index` — an SQLite FTS5 virtual table (or a PostgreSQL tsvector table) — with document type, project, parent and an access tag. Results are ranked with BM25 and returned with snippets. The query language is parsed by Atlas and re-emitted, never passed through raw, which closes FTS injection.

SYNTAX	EXAMPLE	MEANING
Phrase	`"Bluefjord Freight"`	Exact phrase
Prefix	`Okaf*`	Words starting with Okaf
Boolean	`Mercer AND payment`	Both terms (also OR, NOT, parentheses)
Type filter	`type:entity Okafor`	Only entities
Field filter	`amount>50000`	Numeric and date comparisons on indexed fields
Scope	active project / all my projects	Global search is limited to projects you are a member of

Restricted datasets are visible only to roles with restricted access, and sensitive fields are excluded from the index entirely. Searches can be saved; recent queries are kept for 90 days and can be cleared.

Document processing pipeline

STAGE	WHAT HAPPENS
Extraction	PDF via pypdf (per page), DOCX via python-docx (per section), EML (per message part), TXT and Markdown; image-only PDF pages use tesseract + pdftoppm when installed, otherwise the document reports OCR required
Language	Stop-word heuristic for English, Spanish, French, German, Italian, Portuguese and Dutch, with a confidence; others are “undetermined”
Classification	Keyword rules; a label needs at least two cues
Keywords	Top terms per document
Entity candidates	E-mails, phones, dates, money amounts, proper names and gazetteer matches against known entities — always unverified candidates
Chunking	≈ 900-character passages keeping page/section numbers and character offsets
Duplicates & similarity	5-word shingles with Jaccard similarity for near-duplicates; TF-IDF cosine for “similar documents”
Injection screening	Passages resembling embedded instructions are flagged at ingestion and shown to the user

The grounded document assistant

The assistant answers questions only from retrieved passages. Without a local model it works extractively: it ranks passages, quotes them and cites document and page. With a model, passages are wrapped in delimited `<passage>` blocks, the system prompt forbids following instructions inside them, and any answer without valid citations to retrieved passages is withheld. Answers are labelled (for example *direct evidence*), and when the corpus holds no evidence the assistant declines instead of guessing.

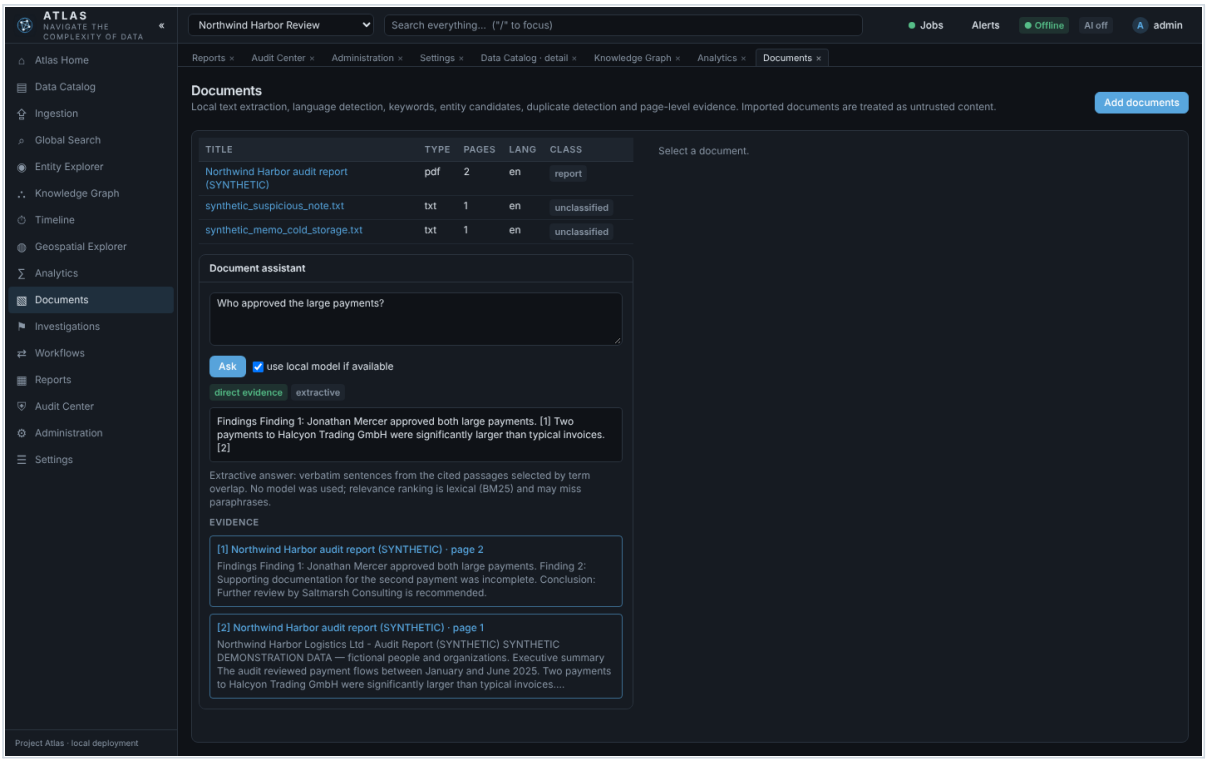


***Figure 12** — Document assistant answering "Who approved the large payments?" by quoting the synthetic harbor audit report, page 2.*

Prompt-injection test built into the sample data

synthetic_suspicious_note.txt contains embedded instructions aimed at an AI assistant. Atlas flags the document at ingestion, marks the passage in answers and does not follow it; the test suite checks both behaviours.

CHAPTER 11

Timeline, Maps and Analytics

11

Atlas answers when, where and how much — and records enough about every analysis that anyone can re-run it and get the same answer.

Timeline

Events are stored in UTC together with the source time text and zone, the record's own created time, when it was ingested and when it was modified — so “when did it happen” is never confused with “when did we learn about it”. The Timeline displays events in the user's chosen time zone, clusters them by a configurable gap, lists relationships active in a time window and compares two entities' activity. Temporal proximity is explicitly described as descriptive, not proof of interaction.

Offline maps and spatial operations

Maps use MapLibre GL with a bundled Natural Earth 1:110m country layer — no tile, glyph or font URLs are referenced. Features are stored as GeoJSON with bounding-box columns for fast filtering. Supported operations: layer toggles, heatmap and grid clustering, time filters, bounding-box queries, radius search and distance measurement (haversine), GeoJSON layer import, annotations and point-in-polygon spatial joins.

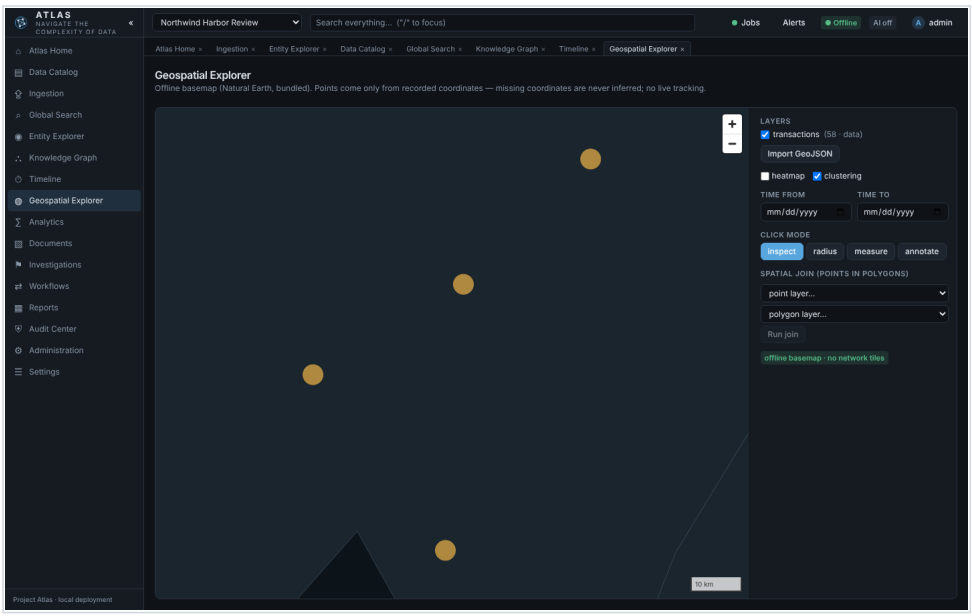


Figure 13 *— Geospatial Explorer over the offline basemap with synthetic transaction points and district polygons.*

Analysis kinds

KIND	METHOD AND OUTPUT
describe	Count, missing, mean, standard deviation, quantiles and distinct values per field
frequency	Value counts with shares
trend	Counts or sums over time buckets
correlation	Pearson r with 95% confidence intervals via the Fisher z-transform; states that correlation is not causation
outliers	Tukey fences (k = 1.5 by default) ranked by robust z (median/MAD); always carries the not-evidence-of-wrongdoing disclaimer
missing	Missing-value counts and shares per field
timeseries	Resampled series for a numeric field over time

Reproducibility by design

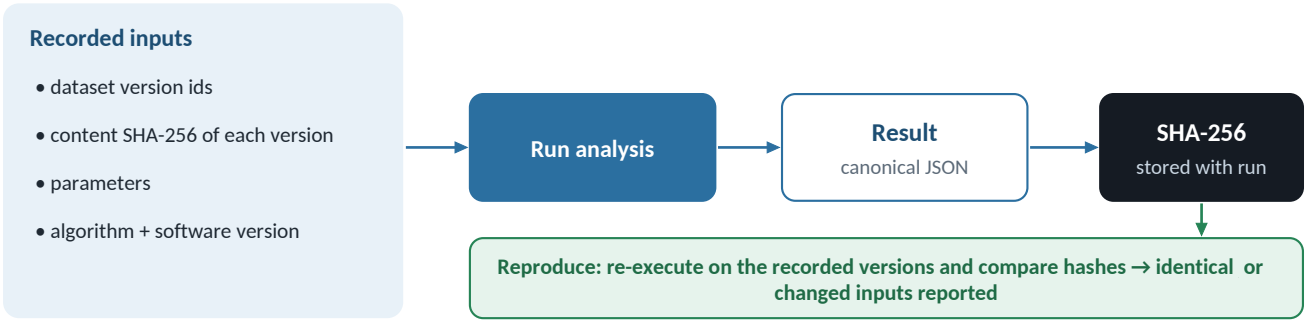


Figure 14 — Each analysis run records its exact inputs and a SHA-256 of the canonical result; reproduction re-executes and compares.

In the acceptance scenario the outlier analysis on the synthetic transactions flags the two planted payments (T-5007 and T-5023) and reproduction returns an identical hash. If an input version has changed, reproduction reports it instead of silently producing a different number.

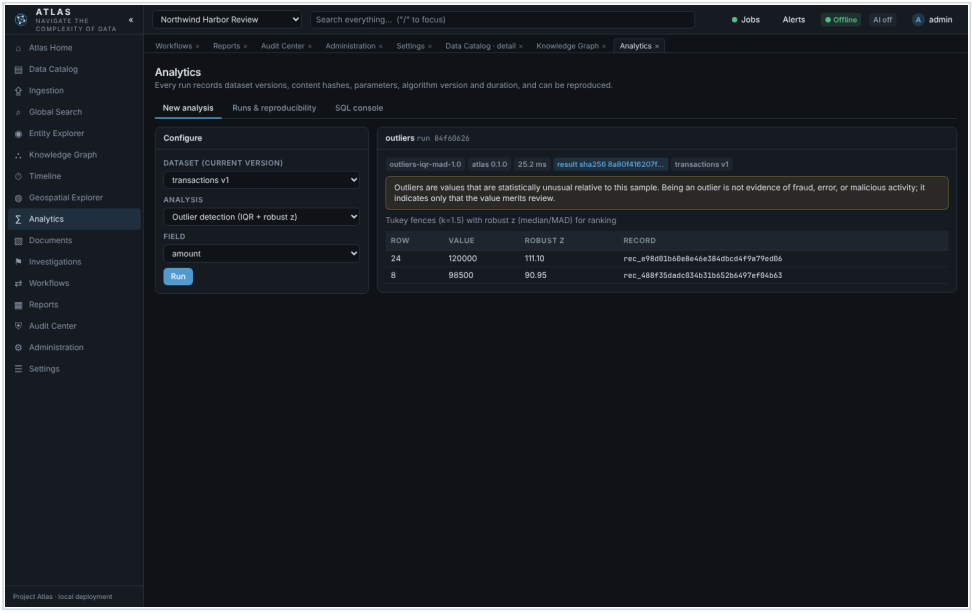


Figure 15 — Outlier analysis with method, bounds, ranked results and the disclaimer.

The sandboxed SQL console

Analysts can query current dataset versions with read-only SQL, for example `select receiver_org_id, count(*), sum(amount) from transactions group by 1`. Four independent layers make this safe even if one fails; a test proves that file access is blocked with the validator deliberately bypassed.

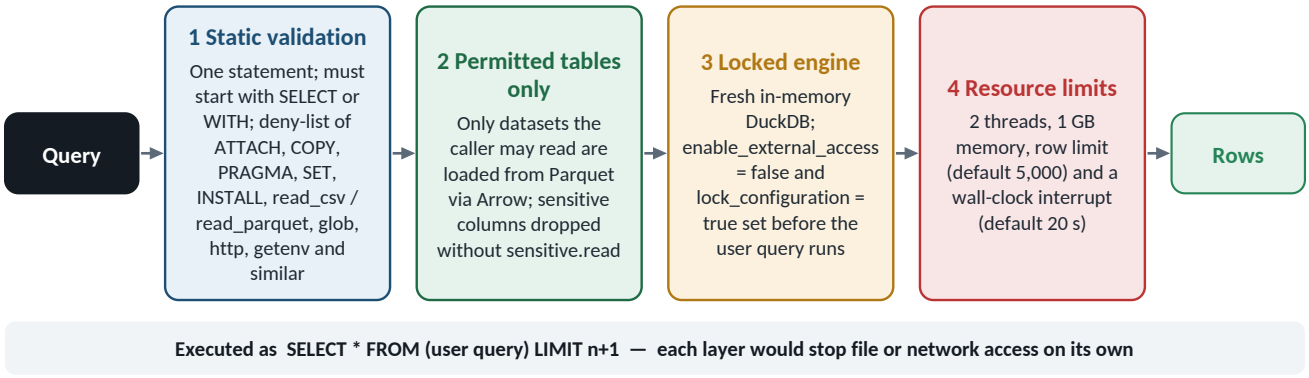


Figure 16 — Defence in depth around the SQL console (analytics/sql.py).

CHAPTER 12

Optional Local AI

12

Every Atlas feature works without a language model. When one is run locally, Atlas applies strict rules: local endpoint only, grounded inputs, enforced citations and a human decision on every output.

Rules enforced in code

- AI is used only when `ATLAS_AI_ENABLED=true` and a registered, enabled, healthy model exists; the UI shows AI features only then and only to roles with *ai.use*.
- `ATLAS_AI_BASE_URL` must be loopback, private or `localhost` in offline mode (checked at startup and on every call). There is **no fallback to any hosted API**.
- Outputs are labelled as model output. Generated SQL is validated and never executed automatically; relationship suggestions are stored as *model_suggested / unverified* and hidden until a reviewer verifies them.

Assistants

ASSISTANT	WHERE	GUARDRAIL
Grounded Q&A, semantic search	Documents	Answers withheld without valid citations; local embeddings, no external vector store
Ask your data	Analytics	Model drafts SQL → user reviews and runs it → deterministic table → model describes it
Schema-mapping suggestions	Ingestion	Names, types and sensitivity validated before applying; marked AI
Quality prioritization, document summary	Catalog, Documents	Labelled model output; deterministic explanations always shown
Relationship hypotheses	Documents	Quote verification; stored unverified; reviewer verifies or rejects
Investigation and report summaries	Investigations, Reports	Cite evidence [E#] or confirmed findings [F#]; invalid citations flagged; inserted only explicitly

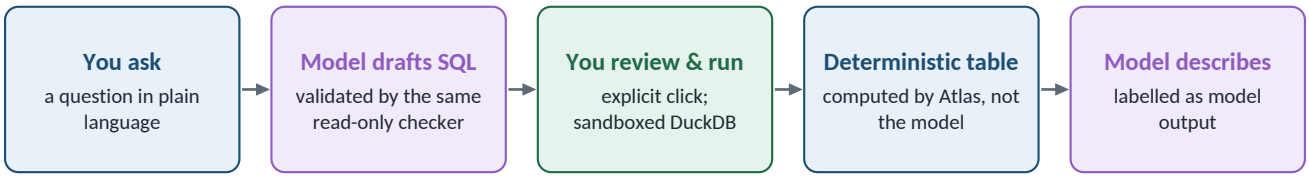


***Figure 17** — "Ask your data" keeps computation deterministic: the model drafts and describes, Atlas computes, and the user decides.*

Testing status

All AI paths are tested against a mock Ollama-compatible runtime — citation enforcement, SQL drafts validated and never executed (including a model-proposed DROP TABLE), unverified hypotheses, embeddings, 503 when disabled — but have not yet been exercised against a real model.

CHAPTER 13

Investigations, Workflows and Reports

13

The final stages turn exploration into accountable output: evidence that cannot drift from its source, findings that another person must confirm, repeatable pipelines and reports that carry their own provenance.

Investigations

An investigation records an objective, a scope or authorization basis and a classification. It is visible only to its owner, collaborators and project administrators. Datasets, entities and graph snapshots can be linked as items, and notes capture reasoning. **Evidence** is captured from records or document passages; excerpts must be verbatim (the API checks them against the source), carry their location (page or row), classification and provenance, and every edit creates a revision.

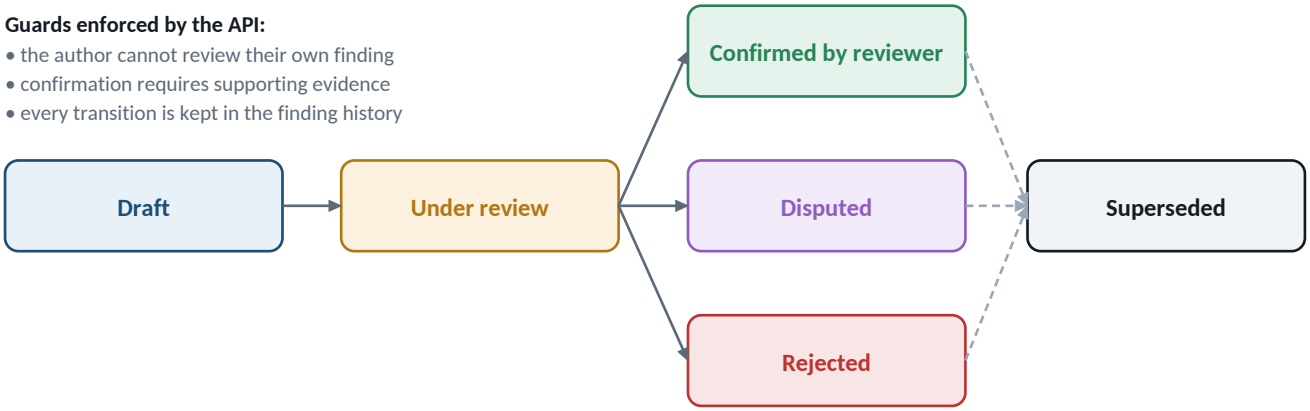


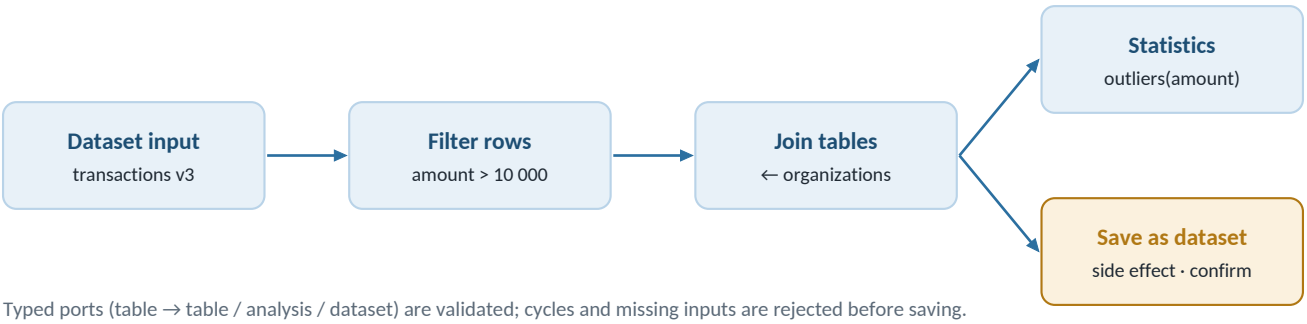
Figure 18 — Findings state machine. A finding links supporting and contradicting evidence and keeps its history.

Workflows

Workflows are typed, versioned pipelines built visually with React Flow. Each node declares input and output port types, parameters, whether it has side effects or writes data, and the permission it requires.

NODE	OUTPUT	NOTES
Dataset input	table	A specific dataset version
Filter rows · Select columns · Normalize text	table	Row predicates, projection, trimming and case
Join tables · Remove duplicates	table	Inner/left joins on keys; dedupe on chosen keys
Quality check	quality	Required violations and duplicate rows
Statistics · Timeline analysis	analysis	Any analysis kind; recorded as a reproducible run
Build graph	graph	Ephemeral graph from table columns (not persisted)

NODE	OUTPUT	NOTES
Entity resolution · Document retrieval · Embedding generation	candidates / passages / embeddings	Reuse platform services
Export CSV · Export report	artifact	Side effect: requires confirmation
Save as dataset version	dataset	Writes data: requires `workflow.write_data`; lineage-linked to the run



Typed ports (table → table / analysis / dataset) are validated; cycles and missing inputs are rejected before saving.

***Figure 19** — A typical workflow: filter large transactions, join organization data, then analyse and materialize.*

Validate rejects cycles, unknown nodes, missing inputs and mismatched port types. *Save* creates a new version. *Preview* executes on at most 200 rows per input, skips side-effecting nodes and persists nothing. *Run* requires confirmation for side effects, reports node results and logs, and rolls back data writes if any node fails.

Reports

Reports are composed from typed sections — summary, methodology, free text, dataset table, analysis, graph snapshot, timeline, map, findings, evidence register, limitations and appendix — previewed, then exported. Charts and diagrams are rendered as vector graphics; PDFs embed DejaVu fonts so any Unicode text renders, and long tables repeat their headers across pages.

FORMAT	NOTES
PDF	ReportLab; vector charts, graph diagrams and scatter plots; page numbers and manifest appendix
HTML	Self-contained; inline SVG charts
XLSX	One sheet per table plus a manifest sheet; formula-injection protection
CSV	Tables only; values starting with `=` `+` `-` `@` are prefixed with an apostrophe
JSON	Full structured content with the manifest

Every export includes a **manifest** — dataset versions and hashes, analysis parameters, software versions, timestamps and any redactions applied — and its own SHA-256. In the acceptance run, the bytes downloaded for a PDF report matched the manifest hash exactly. Users without *sensitive.export* receive redacted exports, and the redaction is recorded in the manifest.

CHAPTER 14

Security Model

14

Security is enforced in the backend for every request, with defence in depth where input is untrusted. The threat model is explicit about what Atlas does and does not protect against.

Threat model

Atlas protects analytical data and evidence on a workstation or private LAN against unauthorized users and roles, cross-project access, privilege escalation, injection through queries, uploads and imported content (including prompt injection), accidental external data egress, and undetected tampering with the audit trail. It does **not** protect against an attacker with full administrative control of the host.

Authentication

CONTROL	IMPLEMENTATION
Password hashing	Argon2id (t = 3, m = 64 MiB, p = 4); automatic rehash on login when parameters change
Password policy	≥ 12 characters, 3 of 4 character classes, must not contain the username, small common-password list
Initial administrator	No default account; one-time random setup token (0600) deleted after use, or atlas create-admin on the host
Sessions	256-bit random bearer tokens, only SHA-256 stored; sliding expiry (default 8 h); revocation per session and on password change, reset or disable
Brute force	10 attempts/min per IP and per username; 15-minute lockout after 5 failures; constant-work verification for unknown users; identical errors; 600 API requests/min per user
MFA	Optional TOTP (RFC 6238, SHA-1, 30 s, ±1 step); administrators can reset a lost authenticator
Recovery	Admin-issued one-time 1-hour reset tokens (stored hashed) delivered out of band; redeeming revokes all sessions

The UI keeps the token in `sessionStorage` and sends it as an Authorization header — there are no cookies, so CSRF does not apply. XSS exposure is reduced by a strict CSP, React's escaping and serving untrusted originals only as attachments.

Authorization: two independent layers

Global roles govern platform administration: System Administrators manage users, projects, backups, settings and models; Security Administrators handle account security, session revocation and audit verification. **System administrators do not automatically see project data** — data access always comes from project membership, keeping administration and analysis duties separable. **Project roles** map to permission sets:

PERMISSION	READ-ONLY	REVIEWER	ANALYST	DATA ENG.	PROJ. ADMIN
Read & search everything permitted	✓	✓	✓	✓	✓
Review matches (entity.review)	·	✓	·	✓	✓

PERMISSION	READ-ONLY	REVIEWER	ANALYST	DATA ENG.	PROJ. ADMIN
Review findings (finding.review)	·	✓	·	·	✓
Run analyses and SQL	·	✓	✓	✓	✓
Investigations, evidence, findings	·	evidence	✓	✓	✓
Reports, workflows, local AI (ai.use)	·	·	✓	✓	✓
Import, ontology, entity resolution	·	·	·	✓	✓
Read restricted datasets	·	·	·	✓	✓
Workflows that write data	·	·	·	✓	✓
Read / export sensitive fields	·	·	·	·	✓
Manage members, delete datasets	·	·	·	·	✓

Resource-level rules

RESOURCE	RULE
Projects	Non-members get 404 for every project URL (existence not revealed); attempts audited as denied
Datasets	Restricted datasets visible only with dataset.read_restricted — applied to catalog, records, search, SQL, analytics, timeline, geo, workflows and reports
Fields	Sensitive fields redacted as [restricted] in records, entity attributes, match evidence, quality values, analytics frames and SQL tables; excluded from search; redacted in exports without sensitive.export
Investigations	Owner, collaborators and project admins only; reviewers cannot review their own findings
Exports	report.export; every export and download audited with format, hash and redactions

Input handling

- **Uploads** — extension allow-list, magic-byte checks (PDF, ZIP-based Office, SQLite, Parquet), NUL-byte rejection for text, size limit, filename sanitization, random storage ids with 0600 permissions, and every stored path resolved and checked to stay inside the workspace.
- **XML** DTD/ENTITY declarations rejected; **SQLite** sources read-only; **exports** protected against formula injection; **queries** parsed or sandboxed (Chapter 11); all database access parameterized through SQLAlchemy.
- **Prompt injection** — imported documents are untrusted: delimited passages, a system prompt forbidding embedded instructions, no tool or action capability, citation enforcement and flagging (Chapter 12).

HTTP hardening and secrets

Every response carries a strict Content Security Policy (`script-src 'self'; connect-src 'self'; frame-ancestors 'none'; object-src 'none'`), X-Frame-Options: DENY, nosniff, no-referrer, COOP and a Permissions-Policy; HSTS is added behind TLS. No credentials are committed: the setup token and checkpoint key are generated at runtime (0600), logs and audit details scrub secrets, and backup passphrases are never stored.

Security limitations, stated plainly

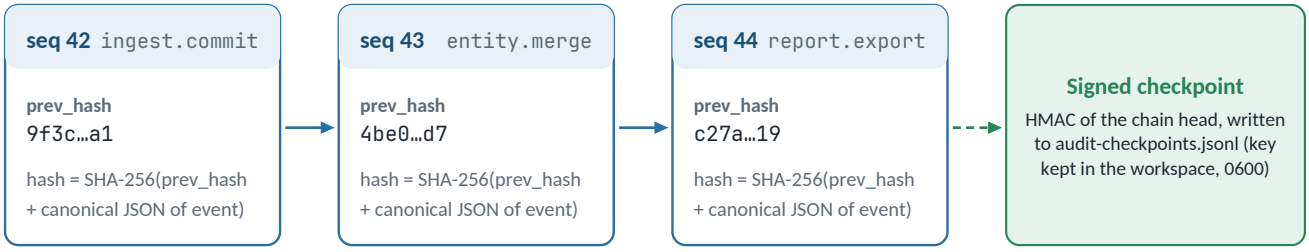
Host administrators can read the database and files or rewrite the audit log with its key; data at rest is not encrypted by Atlas (only backups); rate-limit state lives in memory; sensitive protection depends on labelling; no independent penetration test has been done. Use disk encryption, account separation and off-host checkpoints.

CHAPTER 15

Audit, Backup and Offline Assurance 15

Three mechanisms let an organization trust Atlas after the fact: an audit trail that reveals tampering, backups that are verified before they are needed, and evidence that the process really stayed offline.

Hash-chained audit log



Database triggers reject UPDATE and DELETE on audit_events.
Verification recomputes every hash and reports the first broken sequence number.

Figure 20 — Each event's hash covers the previous hash and the canonical JSON of the event.

Audited actions cover authentication, setup, permission and membership changes, uploads and imports, dataset changes, merges and match decisions, relationship changes, analyses and SQL (including denials), workflow runs, assistant use, exports, backups, restores and AI operations. A test drops the trigger, edits a row and confirms verification reports the exact sequence number.

Backups

ARCHIVE MEMBER	CONTENT
`manifest.json`	Format and software version, schema revision, row counts per table, SHA-256 of every member, encryption flag
`database.sqlite3`	Consistent snapshot via SQLite's online-backup API while Atlas runs; integrity-checked before archiving
`workspace/files, derived, exports`	Uploaded originals, Parquet copies of dataset versions, report and workflow exports
`audit-checkpoints.jsonl`	Signed audit checkpoints
`audit-checkpoint.key`	Checkpoint HMAC key — included in encrypted backups only

Encrypted backups derive an AES-256-GCM key with scrypt ($N = 2^{15}$, $r = 8$, $p = 1$) from a passphrase of at least 12 characters; the whole archive is authenticated, so tampering or a wrong passphrase is detected. Backups can be created from the UI, the CLI or the API, or scheduled with `ATLAS_BACKUP_INTERVAL_HOURS` (newest 10 kept; encrypted via `ATLAS_BACKUP_PASSPHRASE_FILE`). **Verification** checks decryption, safe paths, every member hash, PRAGMA integrity_check and row counts.

Restore procedure

- **Preview** runs full verification and shows the manifest (counts, schema revision, creation time).
- Restore refuses while jobs are running. The current database and workspace are **moved aside** to `.pre-restore-<timestamp>/` — nothing is deleted.
- The archive is extracted, the schema migrated forward if older, live row counts compared with the manifest, and the restore appended to the restored audit chain.
- Tested automatically: backup → delete a dataset → restore → identical counts, working search and SQL, valid audit chain; corruption and wrong-passphrase detection too.

SCENARIO	PROCEDURE
Workstation loss	Install from the offline bundle, start once, create a temporary admin, restore the latest archive (its users replace the temporary one)
Corrupted database	Stop Atlas, keep the damaged files, restore the most recent verified backup, re-import newer data (source hashes reveal duplicates)
Bad import or deletion	Restore into a separate workspace to recover specific data, or in place if losing later changes is acceptable
Suspected tampering	Audit verification reports the first broken sequence; compare with externally stored checkpoints and backups

Offline operation

MECHANISM	DETAIL
No external assets	UI bundle contains all JS, CSS and fonts; bundled basemap; no tile, glyph or font URLs; CSP connect-src 'self'
No telemetry	None is implemented; `ATLAS_TELEMETRY_ENABLED=true` is rejected at startup
Network guard	Socket connect and DNS resolution patched in the atlas-api process; only loopback and private ranges reachable
Local AI only	AI endpoint must be loopback, private or localhost in offline mode; no hosted fallback
Loopback binding	Default `ATLAS_HOST=127.0.0.1`; LAN binding requires explicit opt-in

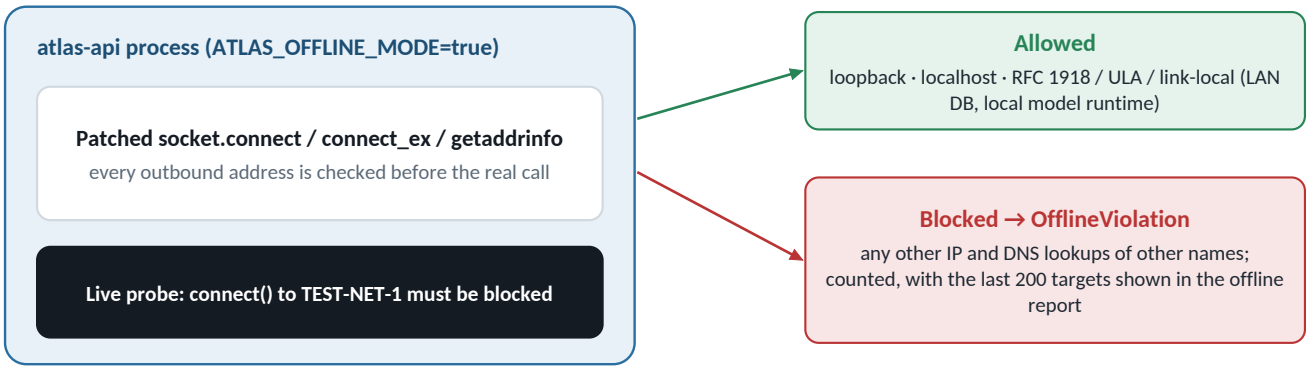


Figure 21 — The in-process network guard (core/netguard.py).

Verification mode (Administration → System & offline, `GET /api/system/offline-check` or `atlas offline-check`) reports offline mode, telemetry, bind address, AI endpoint locality, guard installation, blocked attempts and a live probe. The guard covers only the atlas-api process, so air-gap assurance still needs OS and network controls.

CHAPTER 16

Deployment and Operations

16

Atlas installs the same way on a developer laptop, an air-gapped workstation and a LAN server. Configuration is a set of validated environment variables.

Installation paths

Development checkout

```
cd project-atlas
python3 -m pip install -e backend          # Python 3.11+
(cd frontend && npm ci && npm run build)   # Node 20+; builds frontend/dist
cp .env.example .env
atlas serve                                # http://127.0.0.1:8000
```

Air-gapped workstation

```
scripts/build_release.sh                   # connected host, same OS/arch/Python
tar xzf project-atlas-0.2.0-linux_x86_64-py313.tar.gz
cd project-atlas-0.2.0-linux_x86_64-py313
./install.sh /opt/project-atlas            # checks SHA256SUMS, pip --no-index
/opt/project-atlas/atlas offline-check
/opt/project-atlas/atlas serve
```

Desktop and LAN

`scripts/build_desktop.sh` produces a PyInstaller app that stores data in the platform's user data directory, starts on a random loopback port and prints the location of the setup token on first run (Linux verified; Windows and macOS must be built on those systems). For a LAN server, keep atlas-api on loopback behind the TLS reverse proxy in `deployment/nginx-atlas.conf`, run it with `deployment/atlas-api.service`, set `ATLAS_ENV=production` and `ATLAS_TLS_TERMINATED=true`, and point `ATLAS_DATABASE_URL` at PostgreSQL (preview). On first start, the server writes a one-time setup token; enter it in the browser to create the first System Administrator, or run `atlas create-admin`.

Configuration reference

VARIABLE	DEFAULT	PURPOSE
`ATLAS_ENV`	development	development | test | production
`ATLAS_OFFLINE_MODE`	true	Install the outbound network guard
`ATLAS_HOST / ATLAS_PORT`	127.0.0.1 / 8000	Bind address; non-loopback needs ATLAS_ALLOW_LAN
`ATLAS_ALLOW_LAN / ATLAS_TLS_TERMINATED`	false / false	LAN opt-in; TLS required for LAN production
`ATLAS_DATABASE_URL`	empty (SQLite)	SQLAlchemy URL, e.g. PostgreSQL via psycopg

VARIABLE	DEFAULT	PURPOSE
`ATLAS_STORAGE_DIR` / `ATLAS_BACKUP_DIR`	./atlas-workspace / ./atlas-backups	Workspace files and backup archives
`ATLAS_AI_ENABLED` / `ATLAS_AI_BASE_URL`	false / empty	Optional local model runtime (e.g. Ollama on 127.0.0.1:11434)
`ATLAS_MODEL_DIR`	./models	Where registered model files must live
`ATLAS_SESSION_TTL_MINUTES`	480	Sliding session lifetime
`ATLAS_MAX_UPLOAD_MB`	200	Upload size limit
`ATLAS_QUERY_TIMEOUT_SECONDS` / `_ROW_LIMIT`	20 / 5000	SQL console limits
`ATLAS_GRAPH_NODE_LIMIT`	2000	Maximum nodes per graph view
`ATLAS_BACKUP_INTERVAL_HOURS` / `_RETENTION`	0 / 10	Scheduled backups (0 disables) and how many to keep
`ATLAS_BACKUP_PASSPHRASE_FILE`	empty	0600 file used to encrypt scheduled backups
`ATLAS_RETENTION_ENFORCE_HOURS`	0	Periodic retention sweep (0 = on demand)
`ATLAS_JOBS_INLINE`	false	Run jobs inline (tests, constrained machines)
`ATLAS_TELEMETRY_ENABLED`	false	Must stay false; telemetry is not implemented
`ATLAS_LOG_LEVEL` / `ATLAS_FRONTEND_DIST`	INFO / bundled UI	Logging; override the UI location

Command-line interface

COMMAND	PURPOSE
`atlas serve`	Run atlas-api and serve the UI
`atlas create-admin <name>`	Create the initial System Administrator on the host
`atlas backup [--encrypt]`	Create a backup archive
`atlas verify-backup <file>`	Verify an archive
`atlas offline-check`	Exit code 0 when offline mode is on and the live probe is blocked
`atlas migrate`	Apply database migrations
`atlas-desktop`	Start the desktop experience from an installed package

Common operational issues

SYMPTOM	CAUSE AND FIX
Startup fails with ConfigError	An unsafe combination (LAN without opt-in or TLS, telemetry, non-local AI endpoint); the message names the variable
Project “not found”	You are not a member; Atlas does not reveal whether it exists
Values show [restricted]	The field is sensitive and your role lacks sensitive.read
“database is locked”	A second instance holds the SQLite file; run one instance per workspace
offline-check shows blocked attempts	Something in the process tried to reach a non-local address; targets are listed

CHAPTER 17

Guided Tour: the Northwind Harbor Demo

17

The fastest way to understand Atlas is the synthetic demo workspace. This chapter follows the same path the user guide and the acceptance scenario take.

The synthetic dataset

FILE	CONTENTS AND PLANTED DEFECTS
`synthetic_people.csv`	33 rows / 32 unique people: 1 exact duplicate row, 1 missing and 1 invalid e-mail, 1 null-token phone; each person works for one organization
`synthetic_organizations.csv`	8 organizations (Northwind Harbor Logistics Ltd, Saltmarsh Consulting, Halcyon Trading GmbH, Bluefjord Freight AS …)
`synthetic_contacts.csv`	5 contacts known by name only — the source of the resolution test cases
`synthetic_transactions.csv`	60 transactions: 1 invalid timestamp, 1 latitude out of range, 1 null-island point, 2 statistical outliers (T-5007, T-5023)
Documents (5)	Harbor audit report (PDF, 2 pages), meeting minutes (DOCX), shipment e-mail (EML), cold-storage memo and a prompt-injection note (TXT)
`synthetic_districts.geojson`	4 district polygons for spatial joins

Step by step

#	STEP	WHAT TO DO AND WHAT YOU SHOULD SEE
1	Sign in	Paste the setup token from atlas-workspace/setup-token.txt, choose a username and a strong password. The top bar shows the project, global search, jobs, alerts, the green *Offline* badge and AI status.
2	Create the demo	*Projects → Create demo workspace* builds the project through the normal audited pipelines in seconds: 4 datasets, 106 records, 5 documents, district polygons and match candidates, with live progress.
3	Navigate	Press Ctrl/ K to jump to any page, dataset, document or entity, run actions, or save the current page as a view. *g* followed by a letter jumps between modules; *?* lists shortcuts.
4	Inspect quality	The Data Catalog shows the planted defects: duplicate row and key, invalid e-mail, timestamp and coordinate errors — each with an explanation.
5	Resolve entities	Entity Explorer → Match review shows 2 strong candidates and 1 ambiguous one; merge Dr. Amelia Okafor with Amelia Okafor and note the reason.
6	Explore the graph	Northwind Harbor Logistics Ltd has 4 *works for* edges; run PageRank or communities, then explain the connection between two people.
7	Analyse	Run outlier detection on *amount*: T-5007 and T-5023 are flagged with the disclaimer; reproduce the run and compare hashes.
8	Read documents	Ask “Who approved the large payments?” — the assistant quotes the audit report, page 2; ask about something absent and it declines.
9	Report and audit	Build a report with findings and an evidence register, export to PDF, check the manifest hash, then verify the audit chain in the Audit Center.

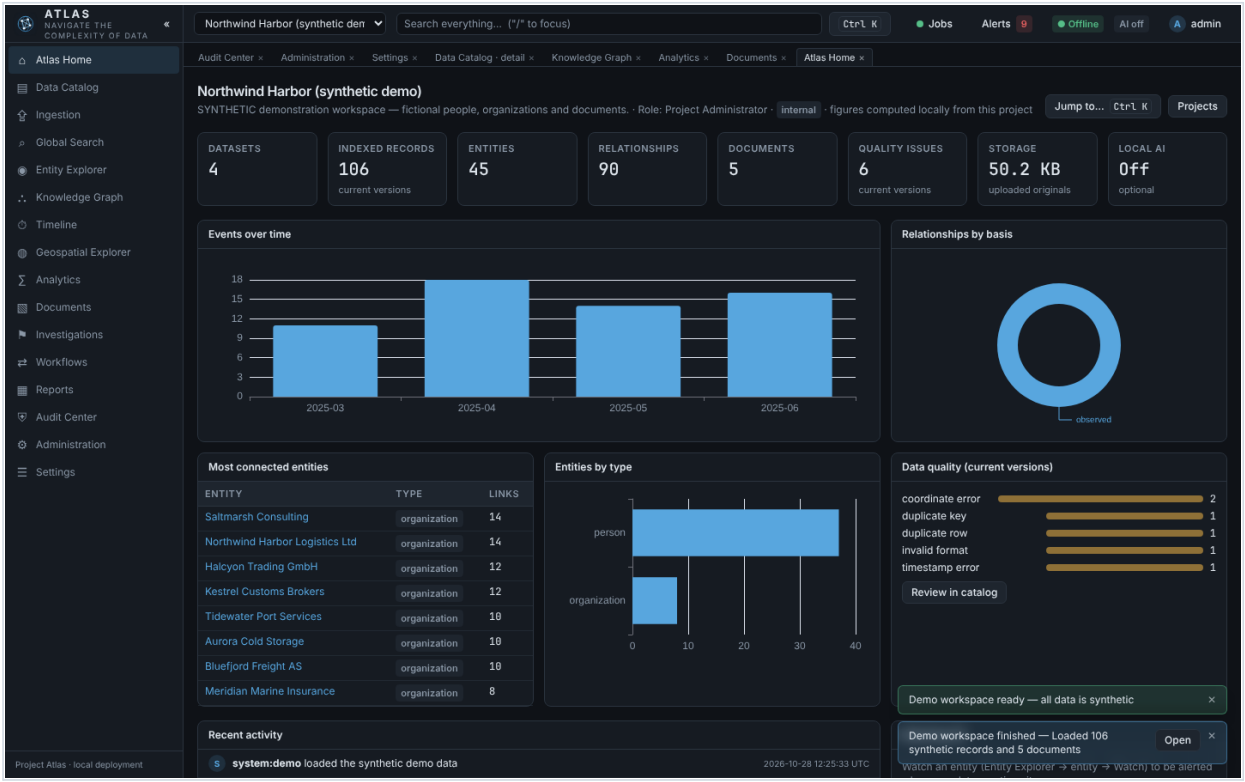
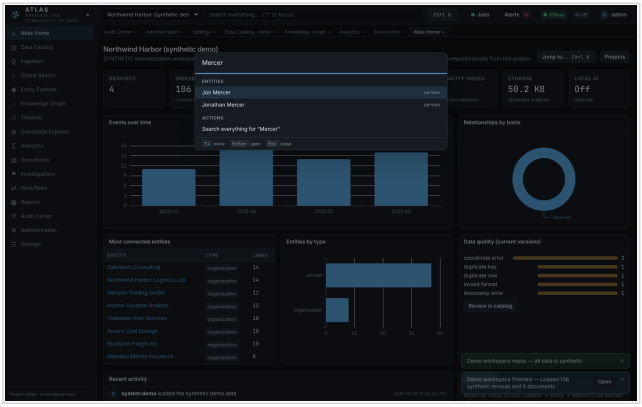
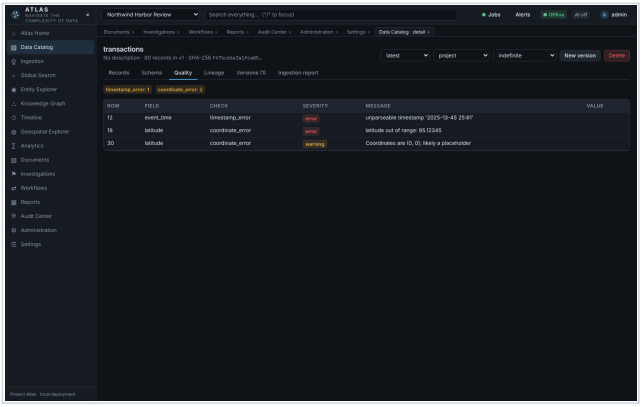


Figure 22 *— Atlas Home after creating the demo workspace: KPI tiles, events over time, relationships by basis, most-connected entities and data quality, all computed locally.*



Command palette (Ctrl/ K): pages, datasets, documents, entities, saved views and actions.



Data Catalog quality tab listing issues by row, field, check and severity.

Figure 23 *— Navigation and catalog views from the demo.*

CHAPTER 18

API Reference Overview

18

Everything the UI does goes through a documented REST API, so Atlas can be scripted, integrated and tested without the browser.

Conventions

- Base path `/api`; interactive documentation at `/api/docs` and the machine-readable spec in `docs/openapi.json` (regenerated by `scripts/generate_api_docs.py`).
- Authentication: `Authorization: Bearer <token>` from `POST /api/auth/login` or `/api/auth/setup`.
- Project-scoped routes live under `/api/projects/{project_id}/…` and return 404 to non-members.
- Errors share one shape, `{"error": {"code", "message", "details"}}`, with HTTP 400, 401, 403, 404, 409, 422, 429 or 503 (AI disabled).
- Long-running work returns a job id; follow it via `GET /api/jobs/{id}` or the SSE stream.

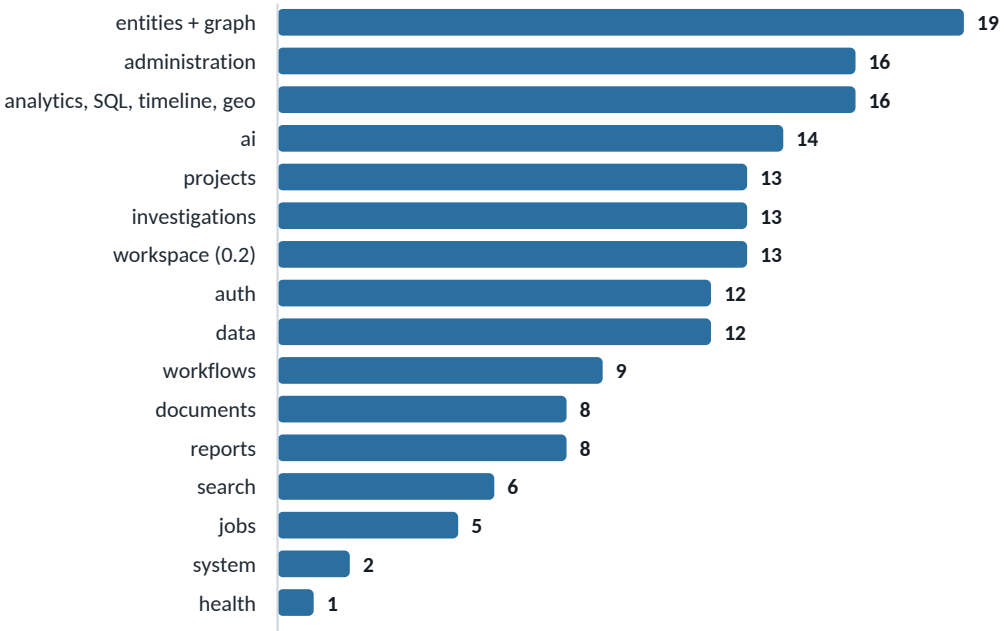


Figure 24 — 167 API operations by group.

Selected endpoints

METHOD & PATH	PURPOSE
`POST /api/projects/{p}/ingest/inspect`	Detect format, encoding, delimiter and types; preview
`POST /api/projects/{p}/ingest/preview`	Dry run over the full file — validation, quality checks, transformed preview; writes nothing
`POST /api/projects/{p}/ingest/commit`	Start the import job
`POST /api/projects/{p}/entities/resolve`	Run entity resolution as a job

METHOD & PATH	PURPOSE
`POST /api/projects/{p}/match-candidates/{id}/review`	Accept (with survivor and reason) or reject a candidate
`POST /api/projects/{p}/merges/{id}/revert`	Revert a merge from its snapshot
`GET /api/projects/{p}/graph/explain`	Shortest paths with per-hop relationships and evidence
`POST /api/projects/{p}/analyses/{id}/reproduce`	Re-execute a run and compare result hashes
`POST /api/projects/{p}/sql`	Sandboxed read-only SQL
`POST /api/projects/{p}/documents/ask`	Grounded document assistant
`POST /api/projects/{p}/reports/{id}/export`	Export PDF/HTML/XLSX/CSV/JSON with manifest
`POST /api/admin/audit/verify`	Verify the audit hash chain
`POST /api/admin/backups/restore`	Preview (confirm=false) or perform (confirm=true) a restore
`GET /api/system/offline-check`	Offline verification report

Example: run and reproduce an analysis

```
POST /api/projects/prj_…/analyses
{"kind": "outliers", "dataset_id": "dst_…", "params": {"field": "amount"}}

→ 200 {"id": "anr_…", "kind": "outliers", "result_sha256": "580dfffb10e2…",
       "result": {"method": "Tukey fences (k=1.5) with robust z (median/MAD) for ranking",
                  "outlier_count": 2, "disclaimer": "…not evidence of wrongdoing…"}}

POST /api/projects/prj_…/analyses/anr_…/reproduce
→ 200 {"identical": true, "result_sha256": "580dfffb10e2…"}
```

Field names in the example are abbreviated for illustration; the full schemas are in docs/openapi.json.

CHAPTER 19

Testing, Validation and Performance 19

Claims in this document are backed by automated suites, a scripted release scenario that includes running with the network removed, and measured benchmarks.

Test suites (release 0.2.0)

SUITE	COMMAND	RESULT
Backend unit, integration and security	`cd backend && python -m pytest`	111 passed
Frontend unit (Vitest)	`cd frontend && npm test`	11 passed
Frontend type check	`npx tsc -b`	clean
UI end-to-end (Playwright, real server)	`frontend/tests/e2e/smoke.mjs`	passed; no console or page errors across all modules
Release acceptance scenario	`scripts/acceptance_scenario.py`	all 14 steps passed (≈ 12 s)
Offline install from release bundle	install + offline-check + serve inside unshare -rn	passed (install 20.8 s)
Packaged desktop app (Linux)	started inside unshare -rn	passed (startup 2.5 s)

What the backend tests cover

AREA	EXAMPLES OF WHAT IS ASSERTED
Authentication	Setup token required and consumed; weak passwords rejected; lockout; identical errors; session revocation; TOTP; single-use reset tokens
Authorization	404 for non-members on every project route; cross-project ids unusable; read-only blocked from writes; no self-escalation; restricted and sensitive data hidden
Injection & malformed input	FTS injection strings; 14 unsafe SQL variants; sandbox with validator bypassed; forbidden types; magic-byte mismatch; XML entity expansion; path traversal; CSV formula injection; prompt injection
Ingestion	Inference; dry run writes nothing; quality counts match planted defects; all formats; cp1252 and mojibake; DST conversion; failed job leaves no partial data
Entities & graph	Ground-truth matches; merge/revert restores exactly; PageRank checked against NetworkX; snapshots and exports; hypotheses hidden until verified
Analytics, time, geo	Planted outliers; reproducibility and changed-input detection; geo never invents coordinates; joins, radius, clustering
Case work & reports	Verbatim evidence; finding state machine; workflow validation, preview, rollback; five report formats with manifests; Unicode PDFs
Platform	Audit tamper detection at the exact sequence; backup → change → restore; encrypted backup failure modes; network guard; 7 unsafe configurations

Release acceptance scenario

#	STEP	RECORDED OUTCOME
1–3	Install, initial administrator, workspace	Package importable; wrong token refused (403); real token consumed and file deleted; project created
4–5	Import and dry run	8 + 33 + 5 + 60 records; dry run reports 1 timestamp and 2 coordinate errors, writes nothing
6–7	Resolve and explore	3 candidates, 2 strong as expected, merge accepted; 4 works_for edges on Northwind Harbor Logistics
8–9	Evidence and analysis	Row hash and source-file SHA-256 match; 2 outliers, reproduction hash identical
10–12	Report, restart, persistence	PDF bytes match manifest hash; new process; counts identical; audit chain valid over 32 events
13–14	Disable network, repeat	Network namespace with loopback only; login, imports, resolution, merge, analysis, assistant, PDF and offline-check all pass

Performance measurements

Measured with *scripts/benchmark.py --rows 100000 --people 5000* on Linux x86-64, 4 vCPU, 15 GB RAM, Python 3.13, SQLite, single run. The workload produced 205,000 records, 5,499 entities and 99,767 relationships in a 623 MB database; peak memory of the whole benchmark process was 1.29 GB.

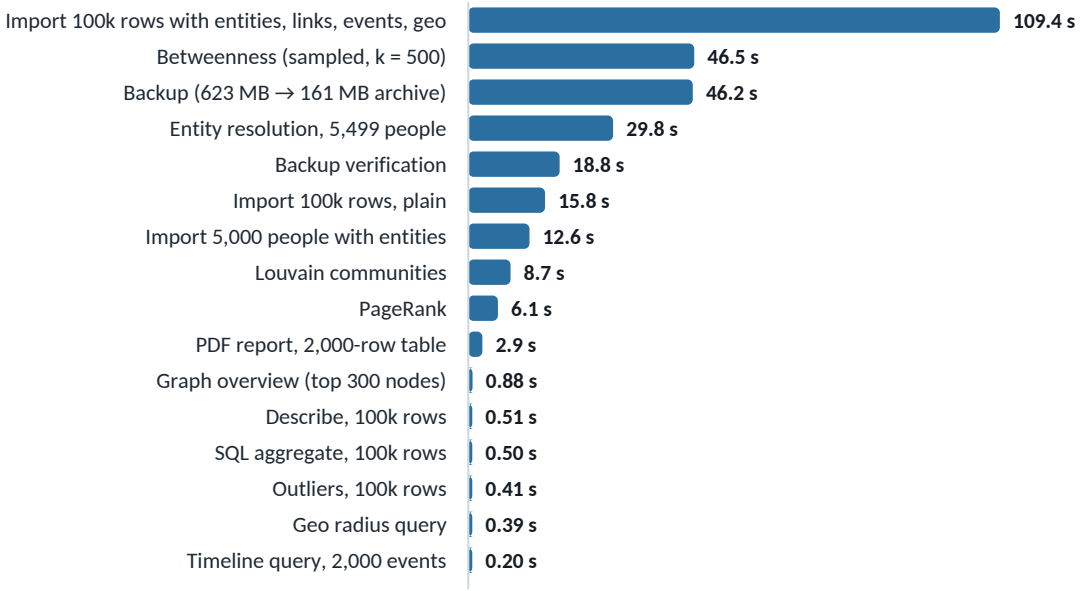


***Figure 25** — Benchmark timings in seconds (linear scale). Full-text search over ≈ 205,000 indexed rows: p50 166 ms, p95 374 ms.*

The measurements drove concrete improvements before release: full import throughput rose from ≈ 350 to ≈ 900 rows/s by batching dependent inserts, deduplicating attribute provenance and adding an indexed FTS rowid map (migration 0003); the graph overview fell from 6.6 s to 0.9 s by ranking degrees in SQL; and a missing SciPy dependency for PageRank was replaced with a verified power iteration. Plain imports run at ≈ 6,300 rows/s.

Sizing guidance (extrapolated, not a certification)

4 cores and 8 GB RAM handle projects of a few hundred thousand records and about 10^5 relationships comfortably. Exact betweenness and Louvain on much larger graphs take minutes — use filters or sampling. Budget roughly 3 KB of database per indexed record plus Parquet copies; no GPU is needed for non-AI features.

CHAPTER 20

Status, Limitations and Roadmap

20

Atlas documents its own gaps. This chapter consolidates what is complete, what is partial, what was deferred and the risks an adopter should weigh.

Completed and tested

Identity and access · projects and membership · permitted data sources · ingestion of 8 formats with inspection, mapping, normalization, dry runs, cancel/retry and progress · 13 quality checks · catalog with versions, lineage, access and retention · ontology editing · entity resolution with review and reversible merges · knowledge graph · unified search · document intelligence and grounded assistant · timeline · offline maps · reproducible analytics · sandboxed SQL · investigations and findings · workflows · reports with manifests · hash-chained audit · backup, verify and restore · offline verification · Linux offline bundle and desktop app · UI for all modules · synthetic dataset with ground truth · 0.2 usability features.

Partial

AREA	WHAT EXISTS	WHAT IS MISSING
Local AI	All assistants with citation and validation checks	Not run against a real model; rerankers unused
OCR	tesseract/pdftoppm path for image-only PDF pages	Untested here; no OCR for standalone images
Language detection	Stop-word heuristic for 7 languages	Other languages report “undetermined”
Tabs	Route tabs, saved views, snapshots, command palette	Tabs are page history, not independent workspaces
Resumable processing	Atomic, idempotent retry from the start	No mid-file resume
Field-level permissions	Sensitive flag with role-based redaction	No per-user or per-field grants
Model permissions	Global admin.models; AI use per project role	No per-project model allow-lists
Workflow editor	Typed nodes, validation, preview, runs, versions	Complex parameters edited as JSON
Retention	Superseded versions purged; owners notified	Current versions never deleted automatically (by design)
Geospatial	Points, polygons, bbox, radius, joins, clustering	Planar point-in-polygon; no line/polygon intersection; 1:110m basemap
Account recovery	Administrator-issued one-time tokens	No self-service recovery (deliberate, offline)

Deferred, and implemented but untested

- **Deferred:** native Tauri/Electron shell; Windows .exe/MSI and macOS builds (must be produced on those systems); a dedicated graph database, external search engine or vector database (not justified at measured scale); multi-process job workers.
- **Implemented but untested:** PostgreSQL deployment (migrations, tsvector search, connection), the PowerShell installer, Windows/macOS desktop launcher behaviour, the nginx/systemd LAN examples and the OCR path.

Outstanding risks

#	RISK	MITIGATION
1	Host administrators can read data and rewrite the audit log with its key	Disk encryption, OS account separation, off-host checkpoints and backups
2	Network guard is process-local and allows private ranges	OS firewall rules, removed interfaces, physical isolation
3	PostgreSQL untested	Validate before multi-user production use
4	SQLite allows one writer; long imports block other writes	Single-user and small teams; PostgreSQL once validated
5	Rate limits and lockouts reset on restart	Reverse-proxy rate limiting on LAN deployments
6	Entity resolution is heuristic; name blocking can miss matches	Identifier blocking; human review of every merge
7	In-memory graph algorithms slow beyond 10^5 edges	Filters and sampled betweenness
8	Sensitive protection depends on labelling	Ontology defaults mark common sensitive fields
9	No independent penetration test	Commission one before high-assurance deployment
10	Dependencies pinned by minimum version only	Release bundles freeze exact versions; rebuild and re-test on update

Suggested roadmap

The following priorities follow directly from the partial, deferred and risk items above. They are recommendations drawn from the engineering report, not commitments recorded in the repository.

HORIZON	FOCUS
Next release	Validate PostgreSQL end to end and add it to CI; run the AI assistants against a real local model; test the OCR path; build and test Windows and macOS packages
Following	Independent penetration test; per-project model allow-lists; finer-grained field permissions; workflow parameter forms instead of JSON
Later	Multi-process job workers; mid-file resumable imports; polygon intersection operations and higher-detail offline basemaps; optional native desktop shell

APPENDIX A

Glossary

A

Terms used throughout this document and the product.

TERM	DEFINITION
Assertion basis	Where a relationship came from: observed, inferred, model_suggested or manual
Blocking	Restricting entity comparisons to pairs that share a name token or identifier, to keep matching tractable
Candidate (match)	A pair of entities proposed by the matcher with a score, band and evidence, awaiting review
Checkpoint	An HMAC-signed record of the audit chain head written outside the database
Dataset version	An immutable import of a dataset with its mapping, schema, quality summary and content hash
Dry run	Full-file validation and quality checking that writes nothing
Evidence	A verbatim excerpt from a record or document passage with location and revisions, attached to an investigation
Finding	A statement in an investigation that moves through draft, review and confirmation with linked evidence
Identity key	A deterministic key derived from identifier fields, used to merge records describing the same entity at import
Manifest	The provenance record attached to every report export: inputs, versions, parameters, software and redactions
Network guard	The in-process patch on socket connections and DNS that enforces offline mode
Normalized value	The typed, cleaned value derived from a raw source value; both are stored
Ontology	The project's configurable entity and relationship types and their fields
Restricted dataset	A dataset visible only to roles with restricted access
Sensitive field	A field redacted for roles without sensitive.read and excluded from search
Snapshot (graph)	Saved node and edge ids, layout and filters for a graph view
Synthetic data	Fictional demonstration data marked `record_origin=SYNTHETIC`
Workspace	A project: its data, members, ontology and work products; also the on-disk storage directory

Source documents in the repository

README.md · docs/architecture.md · data-model.md · security.md · offline-operation.md · installation.md · backup-recovery.md · user-guide.md · troubleshooting.md · api.md and openapi.json · engineering-report.md · release-notes.md · sample-data/expected_results.json. This overview is regenerated with docs/overview/source/build_pdf.py.

APPENDIX B

Frequently Asked Questions

B

Short answers to the questions evaluators ask most often.

Does Atlas need internet access?

No. It installs from an offline bundle and runs with an in-process network guard that blocks every non-local connection. The acceptance scenario repeats the core workflow inside a Linux network namespace with loopback only.

Can it run without a GPU or a language model?

Yes. Every feature works without AI. A local Ollama-compatible model is optional and only adds assistants whose output is labelled, cited and confirmed by a person.

How does Atlas decide two records are the same person?

It does not decide — it proposes. Algorithm er-1.1 blocks on shared name tokens or identifiers, scores only the features both records have, and marks a pair strong only when an identifier agrees and none conflicts. A reviewer accepts or rejects every candidate, and merges can be reverted.

Can an administrator see all project data?

No. System administrators manage the platform but see project data only if they are project members. Restricted datasets and sensitive fields are further limited by project role and enforced by the backend.

What stops someone editing the audit log?

Triggers reject updates and deletes, each event is hash-chained to the previous one, and HMAC-signed checkpoints live outside the database; verification reports the first broken sequence number.

Is the SQL console safe to give analysts?

It accepts single read-only statements, loads only permitted tables into an in-memory DuckDB, disables external access and locks configuration before the query runs, and applies row, memory, thread and time limits.

Can results be reproduced months later?

Yes. Every analysis records the dataset versions and their content hashes, parameters and software versions, plus a hash of the result. Reproduce re-runs it and either confirms an identical hash or reports changed inputs.

How large a dataset can it handle?

On 4 vCPU, 100,000 rows imported in 15.8 s (plain) or 109 s (with entities, links, events and geo), and search over 205,000 rows ran at p95 374 ms. A few hundred thousand records are comfortable on 4 cores and 8 GB.

Which databases and operating systems are supported?

SQLite is the tested default. PostgreSQL is implemented for LAN servers but untested. Linux builds are verified; Windows and macOS packages must be built and tested on those systems.

Is the sample data real?

No. Every person, organization, document and coordinate in sample-data/ is synthetic, generated deterministically with seed 20251027 and marked record_origin=SYNTHETIC; expected results are documented in expected_results.json.